

A Review of the Evaluation System for Curriculum Learning

Fengchun Liu ^{1,2,3,4,5}, Tong Zhang ⁶, Chunying Zhang ^{2,3,4,5,6,*}, Lu Liu ^{2,6}, Liya Wang ^{2,3,4,5,6} and Bin Liu ⁷

¹ Qianan College, North China University of Science and Technology, Tangshan 063210, China

² Hebei Engineering Research Center for the Intelligentization of Iron Ore Optimization and Ironmaking Raw Materials Preparation Processes, North China University of Science and Technology, Tangshan 063210, China

³ Hebei Key Laboratory of Data Science and Application, North China University of Science and Technology, Tangshan 063210, China

⁴ The Key Laboratory of Engineering Computing in Tangshan City, North China University of Science and Technology, Tangshan 063210, China

⁵ Tangshan Intelligent Industry and Image Processing Technology Innovation Center, North China University of Science and Technology, Tangshan 063210, China

⁶ College of Science, North China University of Science and Technology, Tangshan 063210, China

⁷ Big Data and Social Computing Research Center, Hebei University of Science and Technology, Shijiazhuang 050018, China

* Correspondence: hblg_zcy@126.com

Abstract: In recent years, deep learning models have been more and more widely used in various fields and have become a research hotspot for various tasks in artificial intelligence, but there are significant limitations in non-convex optimization problems. As a model training strategy for non-convex optimization, curriculum learning advocates that models learn in the order of easier to more difficult data, mimicking the basic idea of gradual human learning as they learn curriculum. This strategy has been widely used in the fields of computer vision, natural language processing, and reinforcement learning; it can effectively solve the non-convex optimization problem and improve the generalization ability and convergence speed of models. This paper first introduces the application of curriculum learning at three major levels: data, task, and model, and summarizes the evaluators designed using curriculum learning methods in various domains, including difficulty evaluators, training schedulers, and loss evaluators, which correspond to the three stages of difficulty evaluation, training schedule, and loss evaluation in the application of curriculum learning to model training. We also discuss how to choose an appropriate evaluation system and the differences between terms used in different types of research. Finally, we summarize five methods similar to curriculum learning in the field of machine learning and provide a summary and outlook of the curriculum learning evaluation system.

Keywords: curriculum learning; difficulty evaluator; training scheduler; loss evaluator



Citation: Liu, F.; Zhang, T.; Zhang, C.; Liu, L.; Wang, L.; Liu, B. A Review of the Evaluation System for Curriculum Learning. *Electronics* **2023**, *12*, 1676. <https://doi.org/10.3390/electronics12071676>

Academic Editor: Antoni Morell

Received: 1 March 2023

Revised: 21 March 2023

Accepted: 25 March 2023

Published: 1 April 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Optimization for deep learning models has become a research hotspot in various fields, especially non-convex optimization problems, which are considered to be very difficult to solve; there may be an infinite number of local optima in the set of feasible domains, and usually, the complexity of the algorithm for solving the global optimum is exponential. Common optimization methods include stochastic gradient descent, tensor decomposition, etc. Optimization studies of deep learning models have been developed in recent years. In addition to optimization strategies such as changing the network structure and reducing the number of filters, optimization at the data level during training includes minibatch gradient descent, momentum, etc., to obtain faster and more stable convergence by updating parameters for only some relevant samples at a time. In the training process of neural network models, samples are trained in random order, and the samples themselves are of varying difficulty. Curriculum learning [1] sets the order and weight of samples in

the training process according to the difficulty of the samples so that the model can spend less time on noisy and difficult samples in the early stage of training and guide the training of the model toward a better local optimum to achieve a better generalization effect.

The basic idea of curriculum learning originates from curriculum education in human behavior. Human beings need to undergo a long period of training from birth to adulthood, and this training is highly organized, introducing different concepts at different stages, corresponding to a gradual increase in difficulty, and thus gradually mastering the knowledge learned. The concept of curriculum learning was originally proposed by Bengio et al. [1], where the model is trained initially using easier samples and then the difficulty of the samples gradually increases until the entire dataset is utilized for training; they claim that this makes it easy for the model to find better local optima while speeding up the training. Figure 1 shows an example of curriculum learning in animal face recognition. First, the data set is divided into three parts: the first part of the image contains a clean background and objects located in the center of the image; the second part of the image contains multiple objects or a cluttered background; and the third part of the image contains problems such as occluded objects or a cluttered background. The first part of the easy data is used for training in the early stage, and the difficulty of the data is gradually increased until the third part of the difficult data is finally selected for improving the generalization ability of the model.

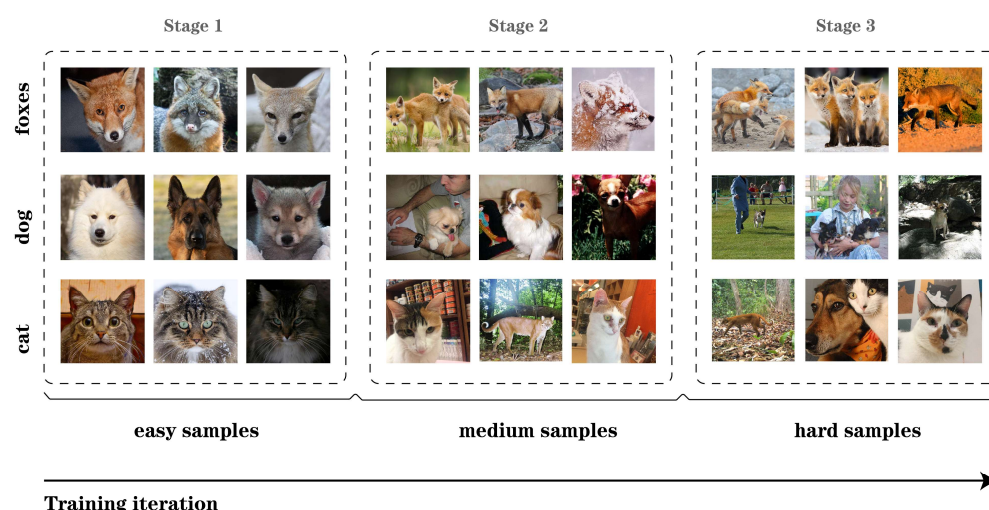


Figure 1. Example of curriculum learning on animal face recognition.

Curriculum learning is widely used in computer vision [2], natural language processing [3], reinforcement learning [4,5], medical diagnosis [6], and cyber security [7,8]. Using curriculum learning methods for model training in a reasonable way can speed up model convergence [3], improve model generalization [9], alleviate data imbalance problems [10], and reduce the negative impact of noisy samples on the model [11]. For example, in eight tasks, including reading comprehension, sentence classification, and similarity analysis [12], the models with curriculum learning all outperformed the normal training models without curriculum learning, with an average performance improvement of 0.9 BLEU points; in neural machine translation [3], the neural translation model with curriculum learning improved by 2.2 BLEU points and reduced the training time by 70%; in the glaucoma diagnosis task [6], the dual-curriculum learning (DCL) reduced the training time by more than half, while it was able to converge to the optimal value stably after about the 20th epoch.

Curriculum learning first needs to evaluate the difficulty of the dataset, realize the sorting or division of samples from easy to difficult, and achieve optimal training through certain training scheduling rules. In this paper, the curriculum learning method is divided into three major stages: difficulty evaluation, training schedule, and loss evaluation. The

involved evaluation methods can be divided into a difficulty evaluator, a training scheduler, and a loss evaluator to form a curriculum learning evaluation system. In summary, this paper makes the following three main contributions:

- (1) Explains the research history of curriculum learning, summarizes its variants and optimization results, and also defines the curriculum learning method.
- (2) Classifies and summarizes curriculum learning research for the three major application levels of data, tasks, and models.
- (3) Offers a comprehensive summary of the methods of the curriculum learning evaluation system, including the difficulty evaluator (evaluating sample difficulty), the training scheduler (establishing scheduling rules based on sample difficulty), and the loss evaluator (evaluating model performance). Provides theoretical support for the application of curriculum learning to various tasks in the field of machine learning.

Based on the summary of the evaluation system for curriculum learning, Section 2 of this paper introduces the basic theory and research history of curriculum learning and summarizes the data, model, and task-level methods for curriculum learning. Section 3 summarizes the difficulty evaluator method in curriculum learning, which is used to evaluate the difficulty of samples and sort or divide the dataset. Section 4 summarizes the training scheduler in curriculum learning, which is designed to establish training rules and select different samples for training in different training periods. Section 5 summarizes the loss evaluator in curriculum learning, which evaluates model performance during training and provides feedback to the difficulty scheduler and training scheduler to optimize model training. Section 6 discusses how to choose the appropriate evaluation system for a specific task and the differences in the terms mentioned between different authors and their research. Section 7 compares and summarizes methodological concepts similar to curriculum learning in the field of machine learning. Section 8 explores a case study of curriculum learning and concludes with a summary of the curriculum learning evaluation system and a discussion of the research directions that exist in the curriculum learning evaluation system that are worth exploring.

2. Basic Theory of Curriculum Learning

2.1. Curriculum Learning Proposal and Development

The earliest ideas of curriculum learning were developed from the study of animal behavior and subsequently applied to the fields of reinforcement learning and machine learning. According to research on curriculum learning, its development can be divided into the stages of conception, proposal, optimization, and integration.

Conception phase. This phase lasted from 1980 to 2008 and began even earlier. The earliest basic ideas of curriculum learning were birthed from animal behavior studies by Skinner et al. [13], who introduced the concept of shaping as a gradual approximation, and subsequent studies showed that shaping accelerates language learning and improves model generalization [14]. The first application of similar ideas to the field of machine learning was proposed by Selfridge et al. [15]. Learning to control problems in physical dynamic systems can be performed by first learning easier systems and then learning the desired system in a series of steps, using a gradual transition from long and light poles to shorter and heavier poles in training cart pole controllers. The use of a sequence in training neural networks from easy to difficult samples dates back to 1993 [16] and mimics human learning behavior. It was proposed that in some cases, neural network models are best trained with easy samples when starting, and a similar approach to curriculum learning was first used in experiments on grammar learning using recurrent networks.

Presentation phase. This phase covers the period from 2009 to 2010. The concept of curriculum learning was first proposed by Bengio et al. [1] in 2009, claiming that training strategies from easy to difficult samples can accelerate training convergence to a global minimum. Viewing curriculum learning as a continuation method for global optimization of non-convex functions, [17] argued that curriculum learning is effective because it can spend less time on noisy and hard to train data in the early stages of training while guiding

training towards better local optima and better generalization. The curriculum in primitive curriculum learning is predetermined by prior knowledge and fixed during training; thus, its high reliance on prior knowledge ignores the progress and feedback of the model during the training process. As a result, Kumar et al. [18] proposed Self-Paced Learning (SPL) in 2010, which designed the curriculum as an SP-regularizer in the objective function of learning and the curriculum is gradually determined by the model itself based on the knowledge already learned.

Optimization phase. In the last decade, with the widespread use of curriculum learning, gradually more and more researchers have focused on the optimization of curriculum learning methods and extended many variants for different tasks. The leapfrog method was proposed by Spitkovsky et al. [19] in 2010, which combined “baby steps” and “less is more” for optimizing unsupervised grammar induction models. In response to the shortcomings of self-paced learning methods, Jiang et al. [20] proposed Self-paced Learning with Diversity (SPLD) in 2014, which takes into account the diversity of samples in the original self-paced learning method and tends to select easy and diverse samples; Jiang et al. [21] proposed Self-paced Curriculum Learning (SPCL) in 2015 for solving the problem that self-paced learning cannot handle prior knowledge and tends to cause the problem of overfitting; and Li et al. [22] proposed Multi-Objective Self-paced Learning (MOSPL) for solving this learning’s initialization sensitivity problem. For the research on the data imbalance problem, Huang et al. [23] proposed the Dynamic Curriculum Learning (DCL) method in 2019 for handling human attribute analysis; Liu et al. [10] proposed the Self-paced Ensemble (SPE) method in 2020 to introduce imbalance learning for solving the problems existing in large-scale, complex, noisy datasets. Zhao et al. [24] proposed Dual Curriculum Learning (DCL) to address the problem of training bias in glaucoma diagnosis caused by category imbalance. In addition, research on improving curriculum learning methods includes Self-Supervised Curriculum Learning (SSCL) [25], Schema-aware Curriculum Learning for Dialog State Tracking (SaCLog) [26], Adaptive Curriculum Learning (ACL) [27], Teacher-Student Curriculum Learning (TSCL) [28], Cyclical Curriculum Learning (CCL) [29], Multimodal Self-paced Learning (MSPL) [30], etc.

Convergence phase. In the last five years, researchers have integrated curriculum learning with other machine learning methods, and many curriculum designs with better results have emerged. Zhang et al. [31] proposed the SP-MIL method combining multiple instance learning (MIL) and self-paced learning (SPL) for saliency detection in 2016, which can alleviate ambiguity in data in the weakly supervised manner of co-saliency detection; Shen et al. [32] proposed Curriculum Dual Learning (CDL) in 2020 by combining dual learning (DL) with curriculum learning (CL) for emotion-controlled response generation; Chen et al. [9] proposed the Curriculum Hardness Aware Meta-Learning (CHAML) framework for the next Point-Of-Interest (POI) recommendation in 2021, integrating curriculum learning into a meta-learning paradigm to address sample diversity in sparse data; Zhang et al. [33] proposed the novel Model Agnostic Meta-Learning (MAML) with curriculum learning to solve individual-level diversity from different moments of a single subject in ventricular arrhythmias based on electrocardiograms (ECGs) in 2022; Morerio et al. [34] combined the dropout method in neural network model training with curriculum learning; Dong et al. [35] combined transfer learning with curriculum learning to propose the Multi-Task Curriculum Transfer (MTCT) method for recognizing detailed clothing characteristics; Tang et al. [36] combined self-paced learning with active learning to address the problem that informative and representative samples in active learning query strategies are not suitable for early stages; Ge et al. [37] combined self-step learning with contrast learning to provide many different forms of category prototypes to provide hybrid supervision; Pi et al. [38] proposed Self-paced Boost Learning (SPBL) to integrate boosting ideas into Self-Paced Learning (SPL) to improve the accuracy and robustness of the model. In addition, it also includes Anti-Curriculum Pseudo-Labeling (ACPL) [39], Curriculum Labeling (CL) [40], Curriculum Pseudo-Labeling (CPL) [41], Transfer Curriculum Learning (TCL) [42], Self-paced Co-training (SPaCo) [43], Meta-curriculum learning [44], SHER [4],

Task auxiliary and Task Difficulty-Hindsight Experience Replay (TATD-HER) [5], Curriculum Learning multitask Classification Attributes (CILCIA) [11], etc. The development of curriculum learning is summarized in Figure 2, and the abbreviations in the figure are described in the text. NLP refers to natural language processing, CV refers to computer vision, RL refers to reinforcement learning, and the rest.

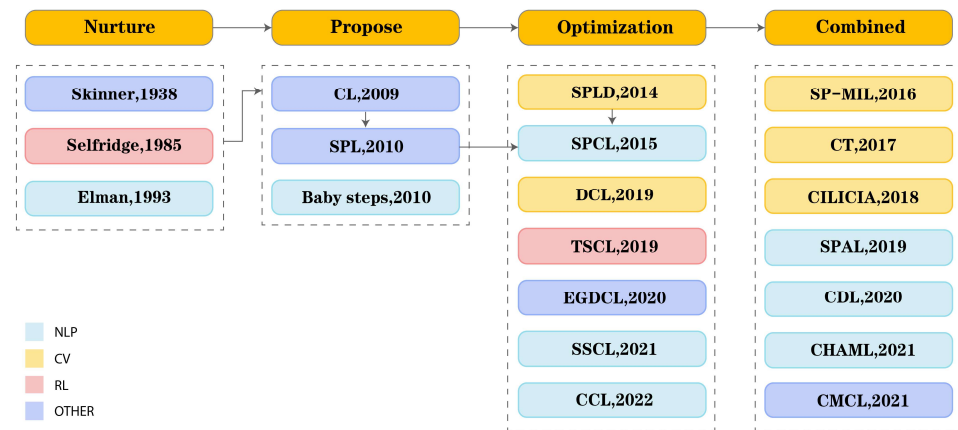


Figure 2. Research history [1,11,13,15,16,18–21,23–25,28,29,31,32].

2.2. Basic Theory

The first curriculum learning approach that emerged was a data-level sampling strategy. As curriculum learning was gradually applied to various fields, many studies emerged. Existing curriculum learning methods can be classified as data-based, task-based, or model-based according to the object of their application.

Data-based. Primitive curriculum learning is a data-level machine learning training strategy that advocates starting with easy samples and gradually progressing to more complex samples and knowledge during model training. Three rules are followed, including a gradual increase in diversity and information (complexity) of the training set, a gradual increase in the size of the training set, and eventually using the entire dataset for training. As research has evolved, researchers have given a broader definition of curriculum learning during its application, allowing it to be applied to more and more domains, such as always training with only fixed-size training sets [45], starting the process with tasks that are highly relevant [46], training from unbalanced to balanced training subsets [23], training from easy to representative samples in sequence [20], etc.

Task-based. Task-based curriculum learning deals with tasks incrementally by focusing on the associations between tasks; each subtask is a simplified version of the next subtask, and each task uses previously learned task knowledge [4,47,48]. In the early stages, the model focuses on easy tasks and gradually shifts to difficult tasks [49]. Metrics of this type of curriculum learning approach include the difficulty of the task [50], the relevance of the task [46,51], and the degree of improvement of the task [28]. For example, in Fu et al. [52], garbage hierarchical classification for cleaning robots, the early model focused on the state of the garbage, the middle stage on the appearance attributes of garbage, and later on specific categories of garbage.

Model-based. This type of curriculum learning method makes the network model achieve better performance by regularly modifying the network model during the training process. Examples include gradually increasing the number of network layers [53,54], controlling filters [55], discarding neuron probabilities [34], and increasing the capacity and strength of discriminators [56–58]. For example, in generative adversarial network models, Karras et al. [53] started with low-resolution images from which the model captures the contour information of the data and gradually adds new network layers dealing with higher resolution details to increase the detail information of the images during subsequent training. Sharma et al. [57] proposed to discover the problem of generators by continu-

ously enhancing the discriminator, which needs to progress under increasingly difficult curriculum tasks to deceive the discriminator and achieve high-quality image generation.

2.3. Method Definition of Curriculum Learning

Curriculum learning as a model training strategy was first defined as a sequence of training criteria C [1], where $Q_t(z)$ is a re-weighting $W_t(z)$ of the original data distribution $P(z)$ and z refers to a random variable in the data set (e.g., (x, y) in supervised learning).

$$C = \langle Q_1, \dots, Q_2, \dots, Q_T \rangle$$

$$Q_t(z) \propto W_t(z)P_t(z), \forall z \in E \quad (1)$$

which satisfies: (i) Gradually increasing the diversity and information of the training subset. (ii) Gradually, more samples are added for training $W_t(z) \leq W_{t+1}(z)$. (iii) Eventually, the weights of all samples are unified and trained on the whole dataset $Q_T(z) = P(z)$. However, with the development of curriculum learning in recent years, more research scholars have extended curriculum learning methods to the task and model levels and even discarded the original three restriction rules. Here we divide curriculum learning into three major stages: difficulty evaluation, training schedule, and model evaluation, which are combined in a general framework according to the objects applied at three major levels: data, task, and model (Figure 3). Let the original dataset be E or task set T , training set e , difficulty evaluator D , training scheduler T , loss evaluator P , model M , a random variable z in the dataset, and a subtask $task$ in the task set.

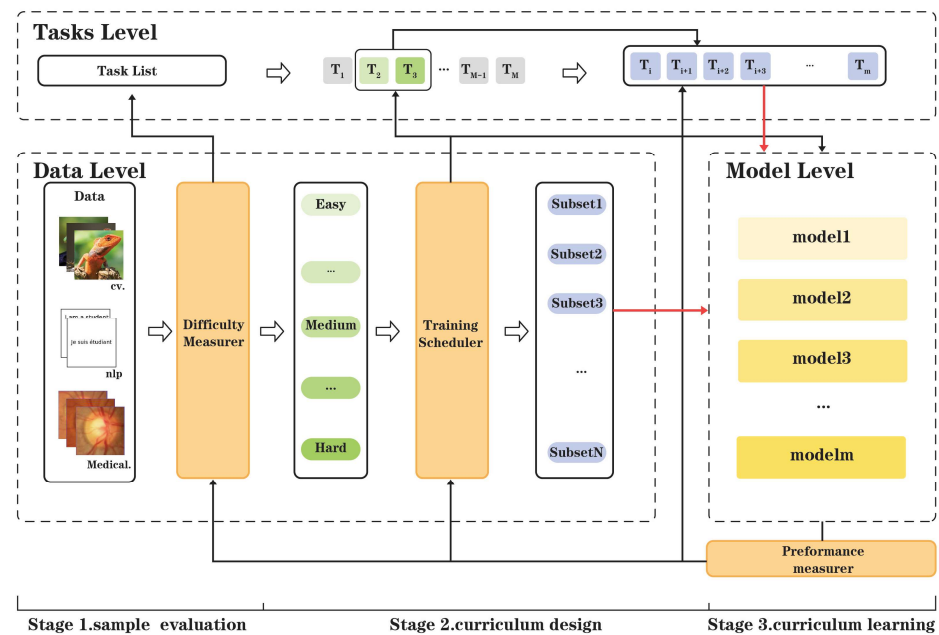


Figure 3. A framework for curriculum learning methods, including three main levels of data, tasks, and models.

Phase 1: Difficulty evaluation. Determine the sample difficulty evaluation metrics for the task, design a difficulty evaluator D to evaluate the difficulty of the samples, and construct a training list L or grouping from easy to difficult, where z_{easy} refers to easier samples and z_{hard} refers to more difficult samples: where the list L is not limited to the order from easy to difficult.

$$L = \{z_{easy}, \dots, z_i, \dots, z_{hard}\}_{i < n}, z \in E \quad (2)$$

Phase 2: Training schedule. Using the training scheduler T to formulate scheduling rules for the model training process, the training subset e_{data} or task subset e_{task} used in the t th iteration of the training process is constructed by sampling from the list L of.

$$\begin{aligned} e_{data}^t &= \{z_1, z_2, \dots, z_m\}, m \leq n \\ e_{task}^t &= \{task_1, task_2, \dots, task_m\}, q \leq n \end{aligned} \quad (3)$$

Phase 3: Model evaluation. Using the training set e generated in the second stage of training, the model learning progress and status are evaluated using the loss evaluator P during the training process and fed back to the training scheduler T and difficulty evaluator D , and the sample difficulty is re-evaluated and the training set e is updated at intervals. This stage involves the curriculum learning of the network structure, following the regular network structure changes, starting from the original model m_1 for training, and gradually modifying the parameters and structure of the network model until the final, complete model M is used for training.

$$\langle m_1, \dots, m_t, \dots, M \rangle \quad (4)$$

Algorithm 1 demonstrates the curriculum learning framework. Step 1: The dataset E is evaluated for difficulty using a difficulty evaluator D to generate a sample list of increasing difficulty. Step 2: The sample list is sampled using the training scheduler to generate the initial training set e . In the iteration number $1 \dots k$ (maximum number of iterations k) period, the current model performance p is evaluated using the loss evaluator P after each iteration. A difficulty evaluator is used to re-evaluate the sample difficulty based on the model performance p to obtain a new sample list l , and a training scheduler T is used to generate a new training set e based on the model performance p and the new difficulty list l (in some methods, the training set is not adjusted at each stage, but is predefined), and the training is continued with the new training set, and the above operation is repeated until convergence.

Algorithm 1: Curriculum Learning Framework

Input: Dataset $E = \{(x_i, y_i)\}_{i=1,2,\dots,n}$; Training set e ; Difficulty Measurer D ; List L ; Training Scheduler T ; Loss Scheduler P ; Model M

Output: the optimal model M^*

```

1:  $L\{easy, \dots, medium, \dots, difficult\} = D(E)$ 
2:  $e = T(L, D)$ 
3: for  $s = 1 \dots k$  do
4:    $e = T(l, p)$ 
5:   while not converged for  $p$  epochs do :
6:     train( $M, e, T$ )
7:   end while
8:    $l = D(E, M)$ 
9:    $p = P(E, M)$ 
10: end for
```

3. Difficulty Evaluator

For curriculum learning, a reasonable sequential list of samples ranging from easy to difficult needs to be constructed first, which involves the problem of sample difficulty evaluation. For tasks in different areas, the sample evaluation metrics used are different due to the variety of datasets and models involved. It is challenging work to find reasonable difficulty evaluation metrics for supporting the effectiveness of the curriculum learning approach for the task. In this section, the design of difficulty evaluators is summarized, which can be classified as heuristic and non-heuristic based on whether they depend on a specific task or not.

3.1. Heuristic Difficulty Evaluator

A heuristic difficulty evaluator is a method to define the difficulty based on human a priori knowledge. This method is achieved by directly judging the difficulty of a sample or by observing the corresponding training data structure. Therefore, heuristic difficulty evaluators are designed differently for different tasks, and this section focuses on the fields of computer vision, natural language processing, and speech processing.

3.1.1. Computer Vision

In the field of computer vision, sample difficulty is defined in terms of attribute category features such as “objectness” and “context-awareness” of unlabeled images [59] or image diversity [31], the number of image labels [60], the degree of image corruption [61], the importance of visual features [62], and image resolution [63,64], etc. In the image difficulty evaluation research by Tudor et al. [65], it was found that the image feature factors affecting the prediction score include the number of categories in the image, the area covered by the most informative category (small objects are more difficult to find), truncation or occlusion, etc. Samples containing multiple object categories and background clutter in a single image have greater ambiguity in the learning process (i.e., are harder to learn), whereas images with clean backgrounds and containing only a single category are easier to learn [66]. For example, objects such as birds and airplanes are both easily detected because their images appear in a single, uniform background of an object, such as the sky. Zhang et al. [60] define the initial image-level curriculum difficulty by counting the number of labels per image, which serves as initial prior knowledge to guide subsequent model learning.

“Objectness” refers to the likelihood that an image region contains a single object of a general category, and “context-awareness” refers to the familiarity with the category of objects surrounding the region. If the system finds the table and the computer monitor first, it has a higher probability of finding the keyboard in between, compared to a lower probability of finding the keyboard if the kitchen object has already been found. Image diversity refers to extensive sampling from multiple groups [21,31], allowing subsequent learning to better take into account objects of different scales, viewpoints, poses, and shapes. For example, Zhang et al. [31] added two prior knowledge—image diversity and spatial smoothness—to self-paced learning to optimize weakly supervised co-saliency detection and eliminate data ambiguity.

In face analysis, metrics such as face image expression intensity level [67] and face size [68] are used as difficulty evaluation metrics. Zhu et al. [68] advocate learning samples of adult faces (easy) at the early stage of model training to provide good initialization for subsequent learning of smaller faces (difficult), and intermediate models learned after samples of adult faces in the previous stage provide a larger effective acceptance domain.

In terms of the degree of image corruption, the corresponding difficulty evaluation varies for different tasks. As in the detection of motion artifacts [61] using the k-space corruption strategy to generate real artificial images for optimal training, where severely corrupted images are defined as easy samples and less corrupted images correspond to difficult samples, the experiments are validated for curriculum, anti-curriculum (from less corrupted images to severely corrupted images) and random curriculum, and the curriculum training model from severely corrupted images to less corrupted images significantly outperforms the remaining two. Instead, images with fewer blurs and smaller cut-outs are labeled as easy samples in image restoration [69], from which the model is trained to obtain a basic representation, and then trained from difficult samples with more blurs and larger cut-outs. In particular, the difficulty evaluation in medical image analysis is based on the degree of disease, such as starting training from images with severe disease (the more severe the image lesion, the easier it is) [70], gradually transitioning to moderate and mild, or starting training from images with nodules [71], etc., or selecting unlabeled images containing high informative content [39] is used to balance the training bias problem in a

medical image, since samples containing high informative content have a higher probability of belonging to a minority class (rare cases).

In addition to this, the difficulty of subtasks or the order between tasks is directly defined without relying on the features of the images themselves as difficulty evaluation metrics. For example, Zhang et al. [50,72] defined learning the global label distribution over images and local distributions over landmarks as an easy task and training the segmentation network as a difficult task in the semantic segmentation of urban scenes, using the results of the easy task to effectively standardize the training of the semantic segmentation network to minimize the domain gap in the semantic segmentation of urban scenes.

3.1.2. Natural Language Processing

In the field of natural language processing, the difficulty evaluation metrics related to sample features include sentence length [3,19,73,74], word rarity [3], paragraph length [75], number of coordinating conjunctions [76,77], sequence length [45,78], the parse tree depth [77], number of various verbal nouns [77], number of anomalous sentences [79], utterance pair similarity [80], and going from a single domain to multiple domains [81]. The heuristic difficulty evaluator is based on intuition, such as starting training with samples of sentence length 1 and gradually expanding to include samples of sentence lengths 1 and 2, which are short sentences that do not represent all grammar but contain enough information needed to depict slightly longer sentences [19]. Tay et al. [75] proposed to evaluate the sample difficulty based on answerability and comprehensibility, where answerability refers to whether the answer is present in the context and comprehensibility refers to the size of the retrieved document; when the retrieved fragment is small, the model can capture the relevant answer information more easily, while when the retrieved fragment is long, the model needs to perform a deeper understanding to find the corresponding original text.

The same research exists in natural language processing to directly define the difficulty and order between subtasks. For example, Lu et al. [82] proposed to first train the model using a simple event substructure generation task for problems in non-semantic metrics and then train the model on the full event structure generation task; Wang et al. [83] guided the model to follow a learning order from the elementary course (transcription) to the advanced course (understanding and word mapping) to force the encoder to have the ability to generate the necessary features for the decoder.

3.1.3. Speech Processing

In the field of speech processing, the signal-to-noise ratio (SNR) [84–86] and speech length [87] are used as metrics for difficulty evaluation. Gradually increasing the SNR is used to improve the generalization ability of the model. Takahashi et al. [88] proposed to use curriculum learning control parameters for training a source separation model. In the first stage, the concealer is trained to generate sounds similar to the carrier audio; in the second stage, it starts hiding information when the concealer starts producing sounds similar to the source, and in the third stage it starts introducing source separation when the decoder learns how to recover information from the source separation.

The difficulty metrics associated with such difficulty evaluators are highly dependent on the dataset itself and often do not generalize to different tasks. Evaluating samples based on intuitive human knowledge or prior knowledge may not always work, and samples that are difficult for humans may be easy for the model to learn. For example, in the response generation [32], the intuitive training model started with unemotional samples (marked as “neutral”) and gradually added emotional samples, which showed poor performance.

3.2. Non-Heuristic Difficulty Evaluator

Non-heuristic difficulty evaluators are generally driven by data-dependent algorithms or models that process the dataset to output the difficulty scores of the samples. These difficulty evaluators are flexible; they do not require human-designed difficulty evaluation metrics. They are not dependent on domain-specific tasks and are not sensitive to the

dataset. Non-heuristic difficulty evaluators can be classified as human annotation, self-scoring, transfer learning, algorithm-driven, and others. Table 1 shows a summary of non-heuristic difficulty evaluators.

Table 1. Summary of non-heuristic difficulty measurer.

Method		Characteristic	Ref.
Self-Scoring		Using the model to output scores	[26,32] [89,90] [17,18]
Transfer Learning		Using external models to guide knowledge transfer	[90–92] [35,46]
Algorithm Driven		Grouping of data by algorithm	[93,94] [11] [89]
Human Annotation		Leverage expert knowledge with high manual workload	[95–97]
Others	Reinforced Learning	Take a series of actions to maximize the cumulative rewards	[98–102]
	Direct Calculation	Direct calculation of certain types of features for sorting	[23,79] [103,104]

3.2.1. Human Annotation

Human annotation refers to the direct acquisition of sample difficulty scores through testers' responses [65,97] or expert annotations [95,96]. In medical image analysis, Wei et al. [95] used the annotation agreement of seven pathologist annotators as the degree of difficulty of histopathological images, defining easy images when the annotation agreement was higher than 6/7 and difficult images when the annotation agreement was lower than 5/7.

The a priori knowledge score proposed by Jiménez-Sánchez et al. [96] uses Cohen's kappa score as an initial difficulty grade, a value used to measure the consistency of clinical experts' opinions on image classification. This type of human annotation method is more widely used in the medical field because the labeling of medical images requires expert knowledge rather than what can be performed based on human common sense. However, this type of difficulty evaluator method requires a large number of subjects to be tested on all samples to have enough information for evaluation, and this part of the work is undoubtedly costly.

3.2.2. Self-Scoring

Self-scoring uses a dataset to pre-train the model to obtain an evaluation model, using the sample as the input to the model and using the information output from the model as the sample difficulty score, whose information includes prediction accuracy [10,18,32], loss [20,25,105], and the degree of contribution to improving the model [98,106]. For example, the predicted probability product per word in the neural machine translation [89]; the sentiment classification accuracy in the response generation [32]; the confidence score of the network calculated for each sample [90]; the cross-entropy in medical report generation [79]; the prediction entropy [96], etc. As proposed by Zhou et al. [107], prediction label flip is used to compute dynamic instance hardness, which proves difficult when the prediction result of a sample changes frequently during the training process.

Most of these evaluation methods are based on a single model outputting sample difficulty scores, whereas methods that use multiple models to output sample difficulty scores [108,109] are more stable and avoid fluctuations in scores due to a particular model being more biased towards a subset of data in a particular category. As in the cross-review [12,26] method, the corresponding golden metrics are used for different tasks to

calculate the difficulty score for each example in the training set, and the dataset is divided into N subsets, and each subset is trained with a separate model. Suppose a sample is selected from the k th subset, and its difficulty is evaluated using the remaining $N-1$ models. The resulting $N-1$ model evaluation scores are summed to give that sample its difficulty scores. The cross-review method can assess the true difficulty of the sample in a more stable way, but neither of the above two types of methods using models to evaluate the sample use expert or prior knowledge. Dai et al. [26] proposed to use a mixed difficulty evaluator based on rules and models, with the model part using a cross review [12] and the rules part using some common features, including dialog turn number, mentioned name entities, and newly added or changed slots, etc., and the two types of scores are combined to obtain a sample difficulty score. Figure 4 illustrates the cross-review method.

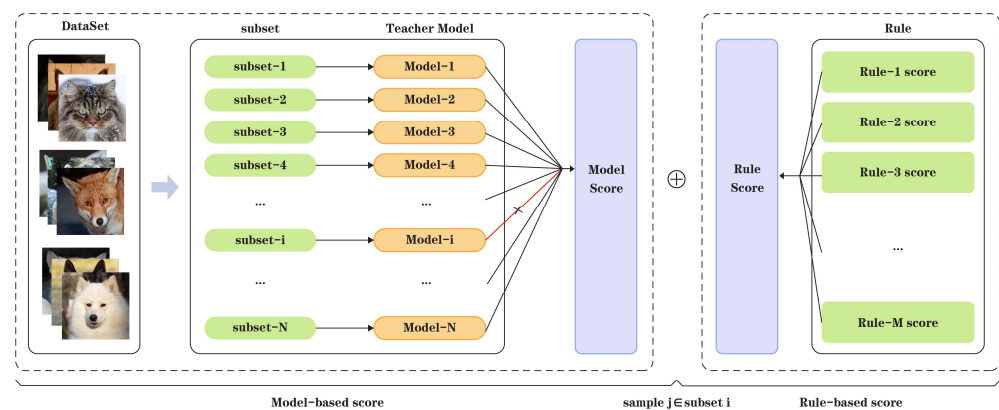


Figure 4. Visualization of the cross-review difficulty evaluator.

In the process of model training, samples with larger losses are harder to learn for the current stage of the model. Conversely, samples with smaller losses prove that the model has been able to correctly predict or classify that sample, which should reduce the sampling probability of that sample. Using sample loss [25,29] as a difficulty evaluation metric, such as Negative Log Likelihood loss [79], square loss [36], and cross-entropy loss [42,110,111] as difficulty scores, is widely used in self-paced learning [18] and its variant methods [22,112]. For example, self-paced learning [18] controls the model to start sampling from samples with smaller losses for training through the coordination of the SP-regularizer and modulates the regularizer to keep decreasing during the training process, guiding the model to gradually sample samples with larger losses. Cross entropy is used as a measure of transferability [42], domain relevance [44], uncertainty [113], and representativeness [36,114], as in Shu et al. [42], where cross entropy loss is used as a measure of sample transferability for solving sample noises of the source domain and distribution shift across domains. In particular, cross entropy is used as a measure of domain relevance in neural machine translation, such as when using the model cross entropy as a sentence divergence score [44]. Where a higher divergence score indicates that the sentence has more in-domain features and is more likely to be different from samples in the generic domain, thus enabling learning from common to individual samples in different domains for better generalization. Zhang [110] and Wang et al. [115] used the cross-entropy of two models for measuring domain relevance and noise level, including the cross-entropy difference between two models trained using out-of-domain data and in-domain data (Moore-Lewis Method) and the degree of change in cross-entropy for selecting general-domain data for model training (Cynical Data Selection) [110], such as assessing the domain relevance of sentences using the cross-entropy of in-domain and general-domain language models [115] (Equation (5)).

$$\varphi(x; \hat{\theta}, \tilde{\theta}) = \frac{\log P(x; \hat{\theta}) - \log P(x; \tilde{\theta})}{|x|} \quad (5)$$

In particular, Mousavi et al. [116] proposed to use two parameters, entropy and mean alpha angle, to obtain direct scattering mechanism information for measuring the degree of complexity of each pixel, which is used to calculate the complexity of each PolSAR image patch.

In practice, research focusing on the instantaneous loss values of samples, as mentioned above, requires evaluating all samples before selecting them at each step, which involves additional inference on unselected samples, and that work is very costly in training. Rather than focusing on the instantaneous loss values of a sample, some studies have focused on its loss value during training, calculating the change in model loss over two consecutive training iterations [107] as a difficulty metric, proving that a sample is very difficult when its loss fluctuates between maximum and minimum values during the sequence. Zhou et al. [117] proposed the exponential moving average (EMA) method for the detection of clean and pseudo-labeled samples. When a sample's loss consistently maintains a low value during training, then its label has a higher probability of being correct, and when a sample's EMA consistency loss remains constant during training, then its pseudo-label is more reliable, achieving the selection of clean, correctly pseudo-labeled data for training and avoiding the inclusion of harmful noisy data. In addition to this, the loss is compared with the threshold value using [18,20,105], and if the sample loss is less than some threshold value, then it is selected as a simple sample, and vice versa, it is defined as a difficult sample.

In the actual neural translation model using pre-training and fine-tuning [118] in training mode, curriculum learning has the limitation that it can only be learned from the beginning, and it would waste computational resources and time if curriculum learning were used to make the pre-trained model learn from the beginning. In the actual training process, all samples cannot contribute equally to the model improvement, and for the model after the pre-training process, most of the samples have been fully learned, and using the same samples for training again may be very small for the model improvement. Under the conventional training cannot further improve the performance of the model, this selection of a subset [98] that has a large contribution or impact on the current model and makes a large change in the performance model is effective and does not require additional new training data. For example, Liu et al. [10] proposed the concept of "classification hardness" for the study of category imbalance, which implies information such as noise, model capacity, and other highly relevant information to the task difficulty. The training model and function is used to give the classification hardness of the samples, which is used to select the training samples with the greatest contribution to the current integration. Also among the five heuristic selection strategies proposed by Sachan et al. [103], including the change in objective (CiO), Expected change in Objective (ECiO), etc., the Expected Change in Objective (ECiO) approach tends to select the problem with the minimum difference between the change in the objective and the expected change in the objective, and this model changes between the expectation and the actual change difference represents the novelty of this problem. This type of difficulty evaluation strategy focuses more on the degree of model change [98,103] or improvement [28,58], using the degree of model change as a measure of how much the current sample has improved the model to achieve the fastest optimal training results. The Genet proposed by Xia et al. [119] automatically searches for environments where the performance of the current model significantly falls behind the traditional baseline solution, and if the current reinforce learning model performs significantly worse than the baseline in the network environment, it proves that the model has a high potential for improvement.

In addition to using the information output from the model itself after pre-training for evaluating the difficult method of the sample, the task model was used as a student role to guide students through the sequence of learning the sample or task using a single [120,121] or multiple teacher model roles [104]. The Teacher-Student Curriculum Learning (TSCL) used by Matiisen et al. [28] in a Partially Observable Markov Decision Process (POMDP) solves the forgetting problem at each period where the teacher instructs the students to

practice those tasks where they make the fastest progress, i.e., where the slope of the learning curve is highest, while selecting tasks where the students perform increasingly poorly (i.e., where the slope of the learning curve is negative); Liu et al. [122] proposed the use of multiple discriminators acting as multiple teacher roles used to guide WGAN training. In addition to this, the collaborative curriculum (CCL) proposed by Huang et al. [123] uses two student networks to regulate each other to remove noisy samples, and the difficulty is evaluated by whether the two student networks choose the sentence with the highest likelihood of conflict. When the sample currently chosen by network A is not the same as the sample chosen by network B with the highest likelihood, the sample chosen by the corresponding network A is marked as a difficult sample.

3.2.3. Transfer Learning

Transfer learning includes model-to-model transfer [90–92] and knowledge-to-knowledge transfer [35,46]. Model-to-model transfer refers to using an external dataset or a small training set to train the transfer model and then transferring the knowledge to the actual model. Or the actual model is obtained by fine-tuning the training set directly on the transfer model pre-trained with external datasets and then using the output of the actual model as the difficulty score. For example, the model is pre-trained with external large datasets (ImageNet, etc.) and then fine-tuned with internal datasets. For example, the network is pre-trained using the entire dataset [90,124], and then a classifier is trained using the output of its activation layer as a feature vector to obtain the confidence of the sample as a difficulty score.

Knowledge is transferred between the intelligence guiding the task, for example, by initially starting training with easy opponents and then gradually increasing the difficulty of the opponents to facilitate the transfer of knowledge and achieve faster learning [125–127]. For example, Pang et al. [126] built 10 difficulty levels of AI in StarCraft II, corresponding to increasing difficulty levels from level 1 to 10, with higher difficulty levels providing less positive feedback, and the paper advocates having agents train at lower levels of AI and then using pre-trained models as initial models for agents to transfer to higher levels.

Knowledge-to-knowledge transfer usually takes advantage of the presence of a correlation between tasks or data to solve tasks in the order of their relevance [46,51], transferring knowledge from a previously learned task to the next task rather than solving all tasks together. An example is the curriculum transfer (CT) method for transferring source annotated knowledge to sparsely labeled target domains [35]. Zhang et al. [128] proposed a two-stage reinforcement learning training model, where the first-stage reinforcement learning agent solves simplified problems and the behavioral cloning technique is used to transfer the knowledge from the first stage to the second stage to initiate strategy training on the original problem. Figure 5 illustrates two types of transfer learning methods. The top panel represents model-to-model transfer, where knowledge is transferred from a model that has been pre-trained through a large public dataset to a model trained from the feature vectors of the pre-trained model. The lower panel represents knowledge-to-knowledge transfer, where knowledge obtained from training in a more relevant task set is transferred to subsequent learning.

3.2.4. Algorithm-Driven

Algorithm-driven refers to the processing of a data set by using an algorithm that outputs information about the sample as a difficulty score. For example, sample evaluation was accomplished indirectly by grouping samples of similar difficulty using general clustering [61], a density-based clustering algorithm [93,94], a hierarchical agglomerative clustering algorithm [11], and Jenks Natural Breaks classification algorithm [89]. This type of method is used for samples whose characteristic attributes are difficult to be evaluated intuitively by a single metric, such as image problems, where the model cannot intuitively evaluate the difficulty score of each image by some single metric. Instead, the algorithm divides samples with similar attributes into the same group, while samples with large

differences are divided into different groups, and the model achieves an easy-to-hard training method by a cyclic sampling of the same group or proportional sampling between different groups. For example, those with high image density distance similarity [93,94] or strong correlation of tasks [11] are classified into the same group by a clustering algorithm, and then the groupings are sorted, etc., or samples are grouped by Jenks Natural Breaks classification algorithm [89], which minimizes the variance within classes and maximizes the variance between classes.

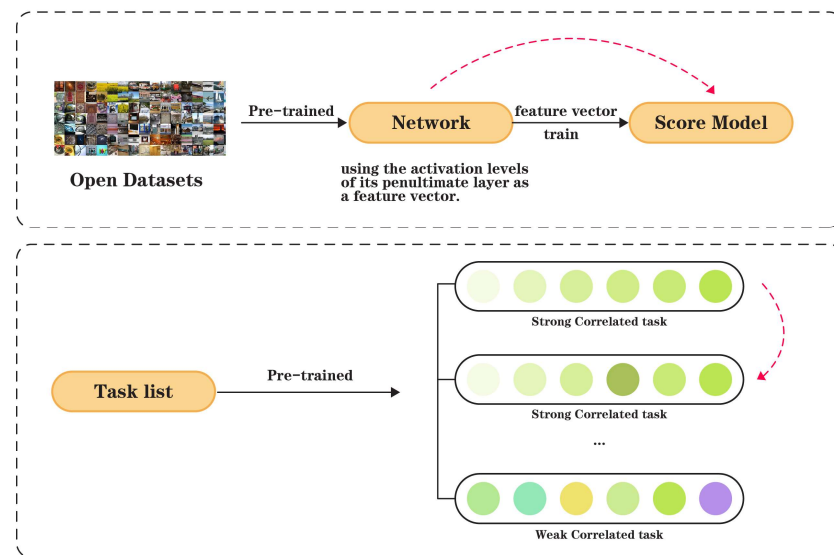


Figure 5. Visualization of transfer learning methods.

The PCDA method proposed by Choi et al. [129] divides the samples into three subsets based on the clustering results, asserting that samples with high-density values are more likely to have correct pseudo-labels, and initially only samples with correct pseudo-labels are used for training. As training proceeds, the classifier can generate reliable pseudo-labels for the remaining denser samples to improve the robustness of the target network. Similarly, the research [93,94] used the clustering of images by projecting each class of images into a deep feature space and calculating the local density of each image, asserting that a set of clean images with correct labels usually have a similar visual appearance and that these images are closely projected in the feature space, leading to a large local density, compared to noisy images that usually have a significant visual appearance, leading to a sparse distribution with smaller density values. This method uses images containing a large number of noisy samples for training the model in an unsupervised manner, which not only allows the model to be trained effectively on large-scale network images and effectively reduces the negative impact of noisy samples on the model but also uses high-noise samples for improving the model's generalization ability through a reasonable curriculum arrangement.

Ge et al. [37] proposed that based on cluster independence and cluster compactness as the cluster reliability index, reliable clusters are represented by the fact that they should have good inter-sample and outside-class distances. Clustering is performed before each iteration round, and only reliable clusters are retained based on the cluster reliability criteria, and the rest of the samples are considered cluster outliers. In addition to this, the Local Style Curriculum Learning (LSCL) approach [130] uses gradient manipulation to produce increasingly difficult adversarial samples. Figure 6 shows the difficult evaluation method of grouping samples based on clustering.

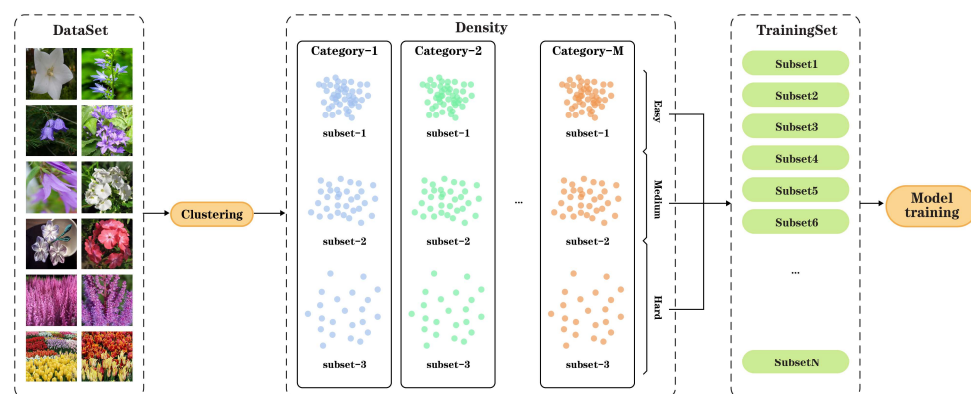


Figure 6. Visualization of difficulty evaluator based on the clustering algorithm.

3.2.5. Others

In addition to outputting sample difficulty scores through models and algorithms, there are also methods to maximize reward [99], maximize learning progress [131], mine difficult samples online [132], and perform direct computation [23,103] for specific evaluation methods. This type of evaluator has the same goal as the aforementioned difficulty evaluator: to design a sample learning order that helps model learning. However, the former is based on human intuitive prior knowledge or an easy-to-hard order directly related to the model, while the latter is a certain order designed for a specific task and does not follow an easy-to-hard order. This type of evaluator, represented by the field of reinforcement learning, uses data selection as the action and model feedback as the state and reward, and dynamically selects sub-tasks for training based on model feedback, with the goal of finding a series of optimal strategies that use the knowledge quickly gained in simple tasks to reduce exploration of more complex tasks [100,101], allowing model performance to be maximized [98]. For example, the process of learning a sequence of edge types is formalized as a Markov decision process in node representation learning for heterogeneous star networks, where the appropriate types of edges are selected for node representation learning by cumulative rewards maximization [99]. The training sequence learns meaningful different types of edges to improve representation learning.

The metrics of a certain class of features are used for ranking by directly calculating them, such as calculating the angle between the question vectors of the feature space [103] for measuring the diversity of the problem; calculating the ratio of samples of different classes to samples of minority classes [23] for measuring the balance of the data distribution; calculating the average cosine similarity between a given image and all normal sample image representations [79,133]; calculating the trace of the transition matrix for assessing the noise level [134], etc. Xiang et al. [104] proposed four calculations for measuring data imbalance in long-tailed data classification, including imbalance ratio, imbalance divergence, imbalance absolute deviation, and Gini coefficient, where imbalance ratio calculates the ratio between the largest and the smallest number of samples, imbalance divergence is defined as the KL-Divergence between the long-tailed distribution and the uniform distribution, and imbalance absolute deviation is defined as the sum of the absolute distance between each long-tailed probability and the uniform probability, etc. In addition to this, Liu et al. [79] proposed to extract the normal image embedding of all normal training images from the last mean pooling layer of ResNet-50 and calculate the mean cosine similarity between the input image and the normal image as an image difficulty metric.

4. Training Scheduler

Curriculum learning in the second stage of model training needs to process the samples whose difficulty scores were evaluated by the difficulty evaluator in the first stage, and a reasonable training scheduler rule needs to be designated for guiding model learning. In

this section, the training scheduler design in the curriculum learning method is summarized, and the training scheduler unfolds according to the three major categories of adjusting time, proportion, and weight. The three data adjustment strategies are not independent, they mostly combine multiple strategies to select samples during the training process.

4.1. Focus on Adjusting the Time of the Sample

The time-based training scheduling method focuses on controlling the time and velocity of adding new samples and controlling when the new samples need to be added to the training set at a reasonable point in time for training. The commonly used scheduling modes can be divided into static and dynamic scheduling. Static scheduling means that the time and velocity of adding new samples to the model are defined in advance, such as through velocity function control and a fixed iteration step, while dynamic scheduling means that the model is adjusted according to its capability or convergence changes during the model training process. Table 2 compares the different methods under static and dynamic scheduling modes.

Table 2. Training scheduler with focus on sample adjustment time.

	Method	Advantages	Disadvantages	Ref.
Dynamic	Model Convergence	Matching with model learning progress	-	[93,94]
	Model Competence	Relate model capabilities to sampling and rationalize the selection of samples that the epochs model can learn	-	[113]
Static	Function	No manual adjustment during training	The speed of adding samples does not match the speed of lifting the model	[3,23]
	Fixed epochs		Difficult to accurately estimate the step length	[91] [90,91]

4.1.1. Static Scheduling

The static scheduling method refers to the fact that the time of adding new samples is predefined throughout the model training process, and the learning ability progress of the model is estimated manually so that the model can learn more efficiently according to its ability and knowledge base at the appropriate training stage. Such adjustment strategies include speed function control and a fixed iteration step size.

- (1) Speed function control. The velocity function directly controls the sampling speed of the model for simple samples through a monotonic nondecreasing function [135], indicating that the proportion of simple samples sampled increases gradually during model learning, with a large slope indicating a fast model learning speed and a small slope indicating a slow model learning speed. In addition, some of the methods that use the model ability function to control the rate of joining samples also use static scheduling by comparing the estimated ability of the model with the sample difficulty scores, and when the difficulty of a sample is less than or equal to the estimated ability, then that sample is included in the training subset for that period; otherwise, it is not included. Since its function involves only the initial sample proportion, the maximum number of iterations, and the current number of iterations [3] factors, the variation of the model capacity is predefined. This type of function control scheduling method, because the speed is predefined, cannot correspond to faster data addition when the model's capability is rapidly improving and may lead to model performance degradation when the corresponding model is improving slowly and data is added too fast. The design of this type of function includes the following:

Linear functions. This type of function introduces new samples at a constant rate during training [3,91]. Where $C_0 \geq 0$ is the initial model capability parameter, such as

C_0 equals 0.01 when the model initially uses 1% simple samples for training. T is the maximum number of model iterations.

$$\lambda(t)_{linear} = \min(1, \frac{(1 - C_0)t}{T} + C_0) \quad (6)$$

Root functions. Root functions tend to improve the ability of the model quickly in the early training phase relative to linear functions, and as training progresses, the sampling of difficult samples slows down [27,32,91]. The short training time for simple samples and the long training time for difficult samples is consistent with the intuition that difficult samples require longer training time due to greater learning difficulty. In general, the later the model samples, the more the difficult samples, and the better the training effect when the parameter p is small. Experiments [3] show that the case $p = 2$ works best. For example, after 125 iterations, the percentage of samples available for $p = 10$ is up to 80%, while $p = 2$ has to be sampled after 600 iterations to reach 80% [91].

$$\lambda(t)_{root-p} = \min(1, \sqrt[p]{t \frac{1 - C_0^p}{T} + C_0^p}) \quad (7)$$

Exponential functions [23,115]. The learning speed of this class of functions varies from fast to slow, $a \in (0, 1)$ is an independent hyperparameter.

$$\lambda(t)_{exponential} = a^t \quad (8)$$

Composite function. A scheduling method was proposed for controlling the distribution of unbalanced training samples from slow to fast and then back to slow [23,135].

$$\lambda(t)_{geometric} = \min(1, 2^{(\frac{\log_2 1 - \log_2 C_0}{T} t + \log_2 C_0)}) \quad (9)$$

Geometric progression function. This class of functions corresponds to a more late sampling of difficult samples [91] and focuses on providing more training time for simple samples.

$$\lambda(t)_{linear} = \min(1, \frac{(1 - C_0)t}{T} + C_0) \quad (10)$$

Other function. Jiménez-Sánchez et al. [136] proposed to rank the samples based on the sample likelihood p^t , where the sample likelihood p^t :

$$p_i^t = \frac{p_i^{t-1} \cdot \exp(\frac{-c_i}{10})}{\sum_{i=1}^N p_i^{t-1} \cdot \exp(\frac{-c_i}{10})} \quad (11)$$

- (2) Fixed epochs length. The training model is divided into M stages by adding new samples after a predetermined number of iterations, and the iteration steps of each stage are determined by the initial sample proportion [90], the maximum number of iterations [91], etc. Three scheduling functions—fixed exponential pacing, varied exponential pacing, and single-step pacing—were proposed in the study [90], where the size of the number of iterations per phase is fixed for fixed exponential scheduling and single-step scheduling, and the size of the number of iterations per phase varies for varying exponential scheduling. Figure 7 shows the visualization of the static training scheduler.

4.1.2. Dynamic Scheduling

The dynamic scheduling method refers to the control of the time when samples are added to the training by calculating the model capability or judging the model convergence during the model training process, including both the model capability and the model convergence.

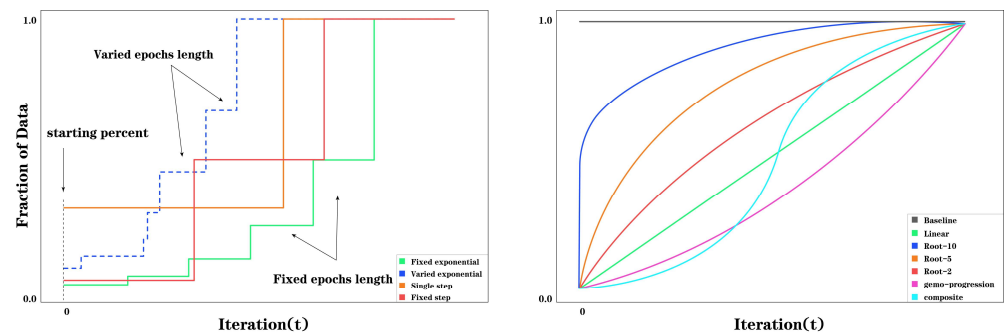


Figure 7. Visualization of static training schedulers. The horizontal axis indicates the number of training iterations, and the vertical axis indicates the proportion corresponding to the data.

- (1) Based on model convergence. When the model has converged in the previous phase or when the model's performance has not improved in a certain period, it indicates that the model has learned sufficiently from the previous training set and a new training set should be added to improve the model's performance. This adjustment strategy is divided into three stages, and in the first stage, only simple and easy-to-learn samples are used for training, allowing the model to learn the underlying knowledge structure of the data from a large number of simple samples and laying the foundation for subsequent learning of more difficult samples, which are mainly low signal-to-noise ratio samples [86], local samples [137], frontal views [138], images containing medium bounding boxes [139], etc. The second stage adds relatively difficult samples for learning, which have mostly noisy labels [93], complex expression and cross-domain samples [20], global samples [137], etc., from which the model can learn more discriminative and meaningful features to improve the model's performance. After the first two stages of learning, the model has sufficient underlying knowledge, and adding difficult samples in the third stage can effectively improve the generalization ability of the model, which is usually unrelated to the attribute classification labels of images, noisy images, etc. For example, Chen et al. [140] used simple images collected by search engines in the first phase of CNN model training for initializing the network and discovering the structure of similarity relationships in the data, and when the model in the first phase converged, difficult images collected on social platforms were used to fine-tune the original network.

In addition to studies that divide the stages of training based on the convergence of the model at the data level, some studies have been conducted from the perspective of regions [22], payloads [141], and embedding rates [142] as updates of the different stages. As in the Multi-objective self-paced learning (MOSPL) proposed by Li et al. [22], a region mixing approach is used, where different stages transition from simple to complex regions to find a reasonable solution path. Figure 8 illustrates the training scheduler approach based on the convergence of the model. The left part refers to the use of more difficult samples in place of the previous training set at each stage of the model training process, while the right part refers to the inclusion of more difficult samples in place of the previous training set by mixing. The orange line refers to the delivery of the model, and the blue line refers to the addition of samples.

- (2) Based on model capability. Based on the relevant parameters used to estimate the potential capability of the model, samples matching the current model capability are selected as the training set for this round of training. When the difficulty of a sample is less than or equal to the ability of the model evaluated in the current training period, the sample will be included in the current training set; otherwise, it will not be included. Section 3.1.1 contains studies related to the static evaluation of model capabilities, with the difference that in this section the model capabilities are evaluated through an adaptive approach rather than a predefined model to calculate

model capabilities, with relevant parameters including the norm [143], the degree of loss reduction [25], and the degree of model improvement [27]. For example, Zhou et al. [113] used the Monte Carlo dropout method to approximate the variance of the network probabilistic distribution given by the Bayesian network as the capability of the model. In particular, Lalor et al. proposed [97] the use of Item Response Theory (IRT) for estimating the ability of deep learning models. Item Response Theory (IRT) is a mathematical model used to analyze performance or questionnaire data by testing a large number of subjects and collecting the graded subject responses that are used to estimate the underlying characteristics of the data. The ability to estimate the model by maximizing the likelihood of a given response pattern and sample difficulty in the research of Lalor et al. is similar to the model being validated against a test set. Table 3 summarizes the two types of model capability estimation methods.

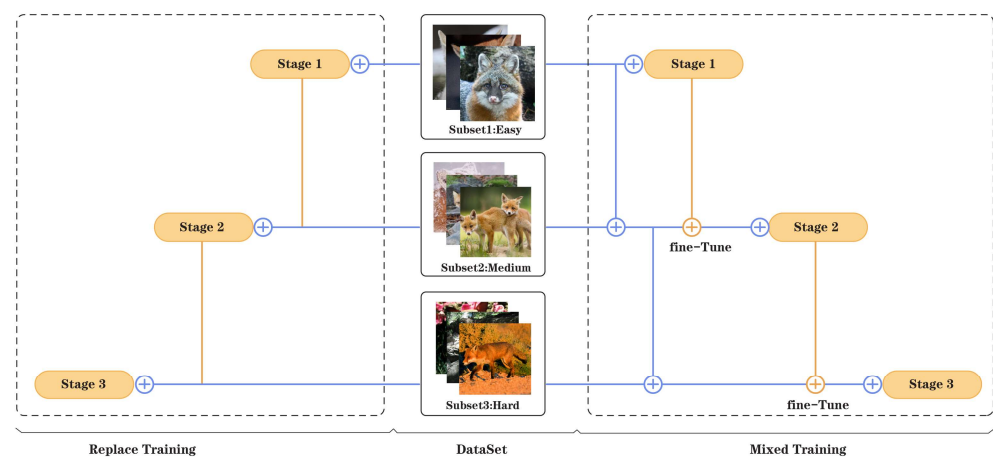


Figure 8. Visualization of a training scheduler based on model convergence.

Table 3. Summary of model competence estimation methods.

	Method	Ref.
Dynamic	Maximizing the likelihood of the data given the response patterns and the sample difficulties to obtain the ability estimate.	[97]
	Use the Monte Carlo Dropout to approximate Bayesian inference, which places a probabilistic distribution over the model parameters on constant input and output data (variance result).	[113]
	function (the parameters include loss reduction/improvement of the model).	[25,27]
	Root function (the parameters include norm/initial value/task-independent hyperparameter).	[143]
Static	Linear/root function (the parameters include the maximum epochs/initial value).	[3,144]

4.2. Focus on Adjusting the Weight of the Sample

The weight-based training scheduling strategy assigns different weights to samples of different difficulties based on the difficulty score, i.e., curriculum learning is used from a probabilistic perspective. The order of the samples is not completely determined, but each sample is given a probability indicating its likelihood of being selected for training, which is adjusted according to the sample difficulty and the number of iterations. Setting higher weights for easy samples at the beginning of training allows the model to learn enough from easy samples while avoiding the negative effects caused by noisy samples or difficult samples at the beginning of training. As the training process proceeds, the weights of the more difficult training samples are adjusted upward, the model learns from the difficult samples to improve its generalization ability, and finally, the sample weights are unified

and trained directly on the complete dataset. Strategies that focus on adjusting the sample weights include direct weighting and threshold weighting.

4.2.1. Direct Weighting

Direct weighting refers to the direct weighting of samples by formula design [124,145], such as by Liu et al. [144], by changing the sampling weights for generic and target domains, so that the model tends to favor generic samples in the early stage of training and gradually learns more complex and higher quality samples as the training proceeds and generates answers with richer and more complete grammars by increasing the sampling probability for samples in the target domain. Zhou et al. [117] used a temperature parameter for measuring EMA loss and EMA consistency loss to implement supervised learning of clean data using correct labels and self-supervised learning of noisy data using reliable pseudo labels.

In addition to this, some studies used external models to complete the weighting of the sample. As in the teacher-student curriculum learning (TSCL) approach proposed by Matlisen et al. [28], the teacher model samples from all tasks early in the model training, and as the student model progresses on a task, the teacher model assigns a higher sampling weight to that task. When the student model has mastered the task, the corresponding learning curve flattens, and the teacher model reduces the sampling weight for that task and assigns high sampling weights to the remaining rapidly progressing tasks until the student model has mastered all tasks and the teacher model returns to a uniform sampling of tasks, with the sampling process simultaneously focusing on tasks with negative slopes of the learning curve. Also proposed by Jiang et al. [146], a teacher network is used to output the weights of each sample during the training of the student model. This type of approach, which uses an external network to guide the model for direct weighting, is driven by the data and addresses the problem of ignoring feedback on model progress.

4.2.2. Threshold Weighting

Threshold weighting first calculates the sample difficulty score and designs either a fixed threshold or a dynamically varying threshold with factors such as the number of sample categories [147,148] and the regularizer [18] for threshold setting. The sample scores are then compared with the threshold values, and different weights are assigned to samples that are on either side of the threshold. Table 4 summarizes the threshold design and the corresponding sample weight assignment design. A more widely used threshold weighting method in curriculum learning is by adding a regularizer as a constraint to the objective optimization function, which was first proposed by Kumar et al. [18] in self-paced learning as follows.

$$(w_{t+1}, v_{t+1}) = \operatorname{argmin}(r(w) + \sum_{i=1}^n v_i f(x_i, y_i; w) - \lambda \sum_{i=1}^n v_i) \quad (12)$$

Self-paced learning is a major branch of curriculum learning, where the sampling of the model is controlled by an SP-regularizer, and instead of designing the scheduling degree manually, the subset of data with the least loss in each iteration is selected by the model for training. The parameters w of a latent variable model are learned by optimizing an objective function. Where $r(\cdot)$ is a regularization function and $f(\cdot)$ is loss function. When λ is small at the early stage of training, the objective function optimization process tends to select samples with small losses and set the weight of this part of the sample to 1. As the number of iterations increases, λ gradually increases and more and more samples are selected, which can be explained by introducing the SP-regularizer into the objective optimization function. The learning coefficient of the SP-regularizer is used as a dynamic threshold to adjust the sample weights, and more samples are continuously introduced through the change of the threshold. For example, a threshold term is added to the objective optimization function in the network embedding [149], and the smaller this threshold is initially set, the greater the probability of simple points being sampled, and as the threshold increases as training proceeds, the greater the probability of complex points

being sampled, until the later stages of training focus on training complex points. Later, Xu et al. [150] introduced privileged information as prior knowledge into the regularizer; Jiang et al. [151] used self-paced learning for multimedia search and proposed linear, logarithmic, and mixture self-paced learning functions; Zhao et al. [105] proposed to extend the weighting scheme to a more effective soft weighted scheme; Li et al. proposed task-oriented [152] and multi-objective-oriented [22,153] weighting schemes. Subsequent studies have proposed to introduce multiple function terms, a negative norm, etc. into the objective optimization equation of self-paced learning and to introduce ease [36], informativeness [36], representativeness [36], and diversity [20,133] into the self-paced learning framework to provide more scheme options for regularizers. Self-paced learning is widely used in various fields, including multi-object ReID [37], target detection [60], matrix factorization [105], co-saliency detection [31], mixture of regressions [154], mixture of regressions [30,151], domain adaptation [42], multi-label learning [155], and network embedding [156].

In addition, in data imbalance studies, the category sample ratio is controlled using a curriculum learning method [23,71], which gradually decreases the sampling of the majority category or increases the sampling of the minority category to achieve a balanced sample distribution. For example, Wang et al. [23] used the category sampling ratio before and after the iteration as a threshold to judge and calculate the weight of that category and continuously increased the sampling of minority category samples to achieve a change in the training subset from a biased distribution to a balanced distribution.

Table 4. Threshold and sample weight assignment design.

	Threshold	Sample Compute	Weight	Ref.
distribution	1	$\frac{D_{\text{target},j(t)}}{D_{\text{current},j}}$	$\begin{cases} \frac{D_{\text{target},j(t)}}{D_{\text{current},j}} \geq 1 \\ 0/1 \end{cases}$	[23]
	$r_{\text{binary}} = -\lambda \sum_{i=1}^n v_i$	$L_i(y_i, g(x_i, w))$	$\begin{cases} 1 & L_i \leq \lambda \\ 0 & L_i > \lambda \end{cases}$	[18]
	$r_{\text{mixture}} = -\lambda \sum_{i=1}^n v_i - \gamma \sum_{j=1}^b \ v^{(j)}\ _2$	$L_i(y_i, g(x_i, w))$	$\begin{cases} 1 & L_i < \lambda + \gamma \frac{1}{\sqrt{i} + \sqrt{i-1}} \\ 0 & \text{otherwise} \end{cases}$	[20]
	$r_{\text{mixture}} = \frac{\gamma^2}{v + \frac{\gamma}{\lambda}}, \gamma > 0$	$L_i(y_i, g(x_i, w))$	$\begin{cases} 1 & L_i \leq \frac{1}{(\frac{1}{\lambda} + \frac{1}{\gamma})^2} \\ 0 & L_i \geq \lambda^2 \\ \gamma(\frac{1}{\sqrt{L_i}} - \frac{1}{\lambda}) & \text{otherwise} \end{cases}$	[105]
regularizer	$r_{\text{linear}} = \lambda(\frac{1}{2}\ v_i\ _2^2 - \sum_{i=1}^n v_i)$	$L_i(y_i, g(x_i, w))$	$\begin{cases} 1 - \frac{L_i}{\lambda} & L_i < \lambda \\ 0 & \text{otherwise} \end{cases}$	[151]
	$r_{\text{logarithmic}} = \sum_{i=1}^n (\zeta v_i - \frac{\zeta v_i}{\log \zeta})$ $\zeta = 1 - \lambda$ $0 < \lambda < 1$	$L_i(y_i, g(x_i, w))$	$\begin{cases} \frac{\log(L_i + \zeta)}{\log \zeta} & L_i < \lambda \\ 0 & \text{otherwise} \end{cases}$	[151]
	$r_{\text{mixture}} = -\zeta \sum_{i=1}^n \log(v_i + \frac{\zeta}{\lambda_1})$ $\zeta = \frac{\lambda_1 \lambda_2}{\lambda_1 - \lambda_2}$ $\lambda_1 > \lambda_2 > 0$	$L_i(y_i, g(x_i, w))$	$\begin{cases} 1 & \frac{1}{m} \sum_{j=1}^m CL_{ij} \leq \lambda_2 \\ 0 & \frac{1}{m} \sum_{j=1}^m CL_{ij} \geq \lambda_1 \\ \zeta(\frac{1}{L_i} - \frac{1}{\lambda_1}) & \text{otherwise} \end{cases}$	[151]
	$l(\mu) = \max(au^2 + b, 1)$	$p_{ij} = \frac{\exp(u_j^T u_i)}{\sum \exp(u_j^T u_i)}$	$\begin{cases} p_{ij} & p_{ij} < l(u) \\ 0 & \text{otherwise} \end{cases}$	[149]

4.3. Focus on Adjusting the Proportion of the Sample

In fact, in a normal machine learning model training process, even without using curriculum learning methods, there are enough easy samples in each small batch for the model to learn. The model can acquire the basic knowledge structure from most of the samples. However, in more difficult problems or tasks, when dealing with noisier datasets, a high proportion of more difficult samples, or when more difficult samples are presented to the model in random order, all of these will result in the model not learning from most of the samples and failing to achieve the expected performance of the model. Therefore, the adjustment of the proportion of samples in different difficulty categories is of critical importance. The methods for adjusting the proportion of samples include thresholding and fragmentation, which are used for model training by adjusting the proportion of samples in each iteration.

4.3.1. Threshold

The difficulty-based sample list is divided by setting a threshold for the difficulty score, starting with the easiest samples first, while the initial proportion of difficult samples is 0. By changing the threshold, more difficult samples are introduced. Thresholds are usually set based on functions (Section 3.1 of this section) or judgments (Section 3.2 of this section), and unlike fragments, samples are introduced for training with consecutive difficulty scores, and consecutive $p\%$ samples from the top of the sample list are taken for training each time. The list is generated based on a single [12,32,109] (Equation (13)), multiple difficulty metrics evaluation [75,79,114,115] (Equation (14)), where samples t , evaluation metrics $C_1, \dots, C_m$, and the metrics weighting factor $\lambda_1, \dots, \lambda_m$. At training epochs t , a batch of training samples is obtained from the top $f(t)$ portions of the entire sorted training samples, including single metric thresholds and multiple metric thresholds. Figure 9 illustrates the threshold-based scheduling strategy.

$$f_{\text{single}}(t) = \lambda C_1 \quad (13)$$

$$f_{\text{multiple}} = \lambda_1(t)C_1 + \lambda_2(t)C_2 + \dots + \lambda_m(t)C_m \quad (14)$$

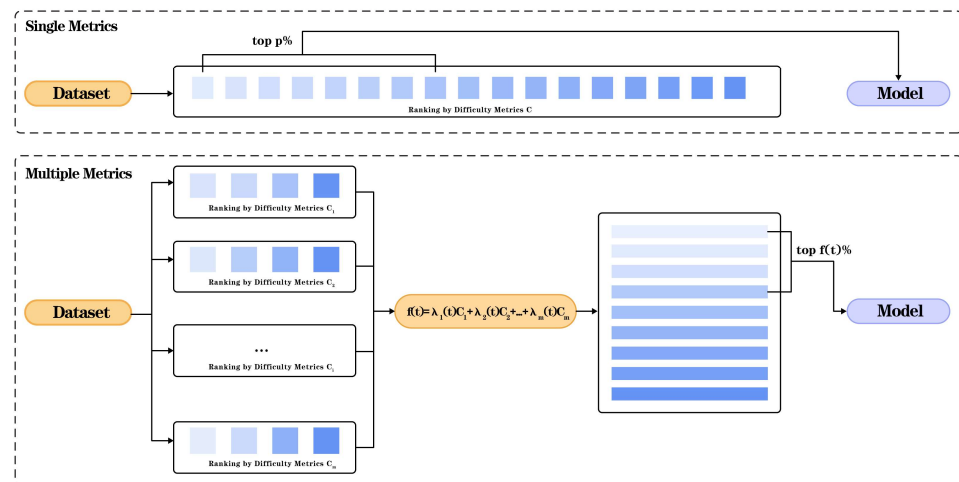


Figure 9. Visualization of a training scheduler based on the threshold.

Single difficulty evaluation refers to the use of only one difficulty evaluation metric to generate a list of sample difficulties, and multiple difficulty evaluation metrics using two or more difficulty evaluation metrics award samples to be ranked from lowest to highest difficulty and then calculate a unified metric $f(t)$ by linear combination, etc. [42,114]. As in PPL [79], the higher the corresponding value, the higher the overall difficulty of that sample. Shen et al. [32] used a simple sample of 1% from the top of the difficulty ranking for training in the initial stage, and this part of the sample contained only one emotion category;

Dou et al. [114] used two types of difficulty evaluation metrics, representative and simple, to dynamically compose the difficulty list and initially used the top $p\%$ of sentences for reverse translation. In addition to considering multiple difficulty metrics simultaneously, Wang et al. [115] proposed the cascaded co-curriculum method to define a scheduling function for domain correlation and noise level metrics, choosing the intersection of the two metrics selected, i.e., keeping only the data selected by both metrics.

In particular, in addition to fixed threshold settings, some studies have focused on transforming fixed thresholds into dynamic thresholds, making the thresholds more compatible with the model's progress. For example, Wang et al. [157] proposed to use reinforcement learning methods to generate a series of dynamic thresholds for selecting reliable pseudo-labeled data rather than based on fixed or manually designed thresholds, taking into account the dynamic capacity of the current model to process pseudo-labeled data with noise, adjusted based on the progress feedback of the model. Zhang et al. [41] gave different thresholds to each class based on the number of samples falling into the class used to reflect the learning effect of the model, and these thresholds were adjusted in real-time with the learning effect of the model.

4.3.2. Fragment

Fragmentation refers to the grouping of datasets based on difficulty scores, where the number of samples in each group is not necessarily the same but the difficulty of the samples within the group is similar, and the scheduling of different proportions of samples is achieved by an adjustment strategy for the grouping. The fragment-based scheduling policies include four types: mixed, single, reversed, and removed. Figure 10 shows a visualization of the fragment-based scheduling strategy.

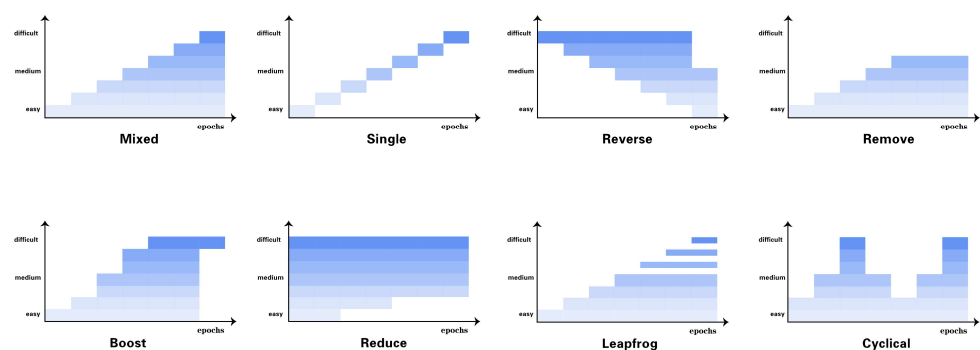


Figure 10. Visualization of a training scheduler based on shards. The horizontal axis indicates the number of epochs, and the vertical axis indicates the difficulty of the sample.

Mixed. This type of adjustment strategy is the standard curriculum learning scheduling approach and is the most widely used training strategy, and the most typical algorithm represents baby steps [19,106]. Model training initially starts with a small proportion of simple samples, while the proportion of difficult samples starts from zero and keeps increasing the proportion [124] until all difficulty category samples are included, eventually adding a stage where the training covers the entire training set [12] to train the model until convergence. This type of adjustment strategy follows the original three conditions of curriculum learning: a gradual increase in the diversity and information (complexity) of the training set, a gradual increase in the size of the training set, and ultimately the use of the entire data set for training [1]. The sampling of easy samples will continue until the end of model training, but since the model has fully learned for easy samples after a period of training and can correctly predict or classify easy samples with a large probability, the model should focus on difficult samples at a later stage, and the continuous sampling of easy samples may cause a waste of computational resources. The use of proportional adoption of different difficulty fragments belongs to a special form of mixing, sampling a certain proportion of fragments from those divided according to their difficulty [12,107,109], providing a natural way of transitioning to

multi-stage learning while avoiding the problem of overfitting simple samples. For example, Liu et al. [10] proposed a hardness harmonize method to divide the majority class samples into k fragments based on “classification hardness”, and equalize the contribution of each bin to “classification hardness” in the initial stage of training, so that the “classification hardness” of samples in each fragment is the same after resampling to emphasize the samples with high contributions, and then use the self-paced factor to reduce the adoption probability of majority class samples to increase diversity. In particular, the cyclical curriculum learning proposed by Kesgin et al. [29] alternates between random training and original curriculum learning during training, with the size of the fragments fixed at $\{0.25, 0.5, 1\}$ cycles of the dataset scale, while the samples of each fragment are resampled based on the probability value of the sample scores, rather than following a fixed difficulty ranked list for selection, with performance better than existing curriculum learning variant models. The steps of the mixed strategy are shown in Algorithm 2.

Algorithm 2: Mixed Algorithm

Input: Dataset $E = \{(x_i, y_i)\}_{i=1,2,\dots,n}$; Training set e ; List L ; Model M

Output: the optimal model M^*

```

1:  $e = \text{sort}(E, L)$ 
2:  $\{e^1, e^2, \dots, e^k\} = e$  where  $L(d_a) < L(d_b)$   $d_a \in e^i, d_b \in e^j, \forall i < j$ 
3:  $e^{\text{train}} = \varphi$ 
4: for  $s = 1 \dots k$  do
5:    $e^{\text{train}} = e^{\text{train}} \cup e^s$ 
6:   while not converged for  $p$  epochs do :
7:     train( $M, e^s$ )
8:   end while
9: end for
  
```

Single. This type of tuning strategy uses only one training subset per phase of training; this training set is divided according to difficulty, and when the performance of the model does not improve by training on the current training subset [45], the next training subset is used for training. For example, Zhang et al. [110] trained in the first phase using only the fragment with the highest similarity score, which is more similar to the data in the domain, and used the next fragment with a lower similarity when that phase of training was over. This type of scheduling strategy is more likely to apply to large data sets, and when dealing with small data sets, the model may replace the training set before sufficient learning has occurred due to too few samples within the fragment or while the training set is updated too quickly. And the model cannot review the previously learned samples in the subsequent learning, which may lead to forgetfulness, resulting in the model’s performance not reaching the expected level. Instead of following a single model for training with a single metric, Tay et al. [75] prioritized answerability metrics and then considered exchanging a simple subset of understandability for training when the use of a simple subset of answerability metrics for training failed to improve the performance on the validation set. The steps of the single strategy are shown in Algorithm 3.

Algorithm 3: Single Algorithm

Input: Dataset $E = \{(x_i, y_i)\}_{i=1,2,\dots,n}$; Training set e ; List L ; Model M

Output: the optimal model M^*

```

1:  $e = \text{sort}(E, L)$ 
2:  $\{e^1, e^2, \dots, e^k\} = e$  where  $L(d_a) < L(d_b)$   $d_a \in e^i, d_b \in e^j, \forall i < j$ 
3: for  $s = 1 \dots k$  do
4:   while not converged for  $p$  epochs do :
5:     train( $M, e^s$ )
6:   end while
7: end for
  
```

Reverse. Reverse refers to the scheduling strategy of anti-curriculum learning [84,89,110,135], where the model initially chooses the most difficult samples to start training [158], forcing the model to learn the more difficult samples earlier and faster, gradually introducing easier samples, and eventually training on the entire dataset. For example, the ACCAN-reversed method in speech recognition extends from high to low signal-to-noise ratios [86]. This scheduling strategy is counterintuitive. When the model is initially trained with difficult samples, it is under too much learning pressure and may not achieve the expected performance. In contrast, when the amount of data is sufficient or the model is relatively stable, such as when the model has been pre-trained [98] or when the model itself is not prone to overfitting or underfitting, it is feasible to initially train with difficult samples, and as easier samples are introduced, the number of difficult samples trained increases, which is beneficial to the performance of the model. In a study by Florensa et al. [158], it was proposed to have the robot gradually learn to reach the goal from a set of starting states that are increasingly far from the goal, achieving effective training of goal-oriented tasks.

Remove. This type of adjustment strategy makes the model focus on more difficult samples by removing easy samples during model training [107] or by removing some of the samples to improve the robustness of the model to certain missing patterns [159]. As in Zhang et al. [160], 20% of the easy samples of the current sample are reduced in each of the three phases of the model training process, i.e., 100%, 80%, and 64% of the samples are used for training in the training process. Kocmi et al. [76] proposed to reduce samples only when converting to higher-complexity fragments, initially sampling from the easiest fragment until there remains the same number of samples as in the second easiest fragment. Then continue to sample uniformly from the first two easiest fragments until each fragment has the same number of samples as the third fragment, etc.

In addition to this, there are some studies that use fragment scheduling strategies such as Boost, Reduce, and Leapfrog. The Boost strategy improves model performance by repeated training on difficult samples [160], for example, uses 10% of difficult sentences for repeated training in the late stage of training. This type of strategy prefers to repeatedly train difficult samples to promote model optimization rather than spend more time on simple samples. Intuitively, the more time the model takes to learn difficult samples, the more steps are required, while repetition for difficult samples does not require recalculating the difficulty factor or adding additional datasets. The Reduce strategy [19], which is the opposite of the Boost strategy, makes the model stop training from easy samples up to a certain difficulty point by removing some of the difficult samples, such as at the knee point of maximum curvature between the rapid improvement of the model and the start of convergence, where the samples are neither too difficult (excluding very difficult samples) nor too easy (providing sufficient knowledge). The leapfrog strategy proposed in the study [19] is trained in the same way as the Reduce strategy in the early stage, starting at special difficulty points and using step lengths to achieve partial sampling of difficult samples, allowing the model to converge earlier to improve training efficiency and avoiding the problem of reduced generalization ability caused by completely discarding difficult samples. For example, the initial training starts with samples of sentence length 1, then adds samples of sentence length 2, gradually adds samples of sentence length 15, and subsequently trains only on samples of sentence length {15,30,45}.

We note that the size of each fragment is critical to the effectiveness of curriculum learning [29]. When the number of samples within each fragment is evenly distributed [76], it may lead to differential fluctuations in samples within the same fragment, i.e., it may be that the difficulty of samples within a fragment is not always similar; there is not enough variability between fragments [89]. In the study [113], dividing the training corpus into four parts worked best to avoid the problem of overfitting the model due to too-small fragments. In contrast, it is more reasonable to divide the fragment size based on sample difficulty [110], i.e., the number of fragments is not the same within the fragments, but there may be too much variation in the number of samples between fragments, and the

learning time needs to be re-examined for each fragment. In particular, the fragment-based proportions scheduler will randomly disorder the samples within fragments before introducing new fragments to avoid overfitting and promote convergence [76,110], i.e., the difficulty between fragments is ordered while the samples within fragments are disordered, helping to increase the uncertainty of the samples within fragments.

5. Loss Evaluator

A loss function or cost function is a function that maps the values of a random event or its associated random variables to non-negative real numbers to represent the risk or loss of that random event. In machine learning applications, the loss function is often used as a learning criterion associated with optimization problems to solve and evaluate models by minimizing the loss function. The third stage of curriculum learning is the loss evaluation of the model progress during training using a loss evaluator [27,69,85], which provides feedback to the difficulty evaluator and the training scheduler to dynamically adjust the learning sessions. At the end of each training phase, the performance of the current model trained on that subset of data is calculated via the validation set. As Gan et al. [25] evaluate the model's performance during the model training process, the decreasing value of the loss is used to measure the competency of the model, and then the results are fed to the training scheduler, making the model select the appropriate training subset at different periods. In particular, when multiple expert models guide a single student model for training, Xiang et al. [104] perform performance evaluation on the validation set at the end of each training phase, and instead of simply summing the losses of the expert models, the TOP-1 accuracy on the validation set is used as a measure of the gap between the expert and student models, and the final knowledge distillation loss is an automatically weighted sum of the knowledge distillation losses of all expert models.

The same idea of curriculum learning ordering can be used in the design of the loss function. The curriculum learning term is added to the loss function to control the model sampling coefficients to achieve data selection adjustment of the model. Zhao et al. [6,24] proposed dual-course learning by encoding sample importance and feature importance into a loss function that is used as a weighting factor to control model sampling, enabling easy to difficult, unbalanced to balanced learning, and allowing the model to focus its training on hard-to-classify, rare-case samples at a later stage. Huang et al. [132] proposed a loss function containing positive and negative cosine similarity modulation for adaptive modulation model training. In the early stage of model training, the value of the modulation function is less than 1, at which time the weight of difficult samples is reduced and the simple samples are emphasized accordingly. As the training proceeds, the modulation function will be greater than 1, when the difficult samples are emphasized. This is in addition to curriculum loss (CL) with a tighter upper bound on 0–1 loss [161], and noise pruned curriculum loss (NPCL) dealing with label corruption [161].

In particular, studies exist that use curriculum learning for the output and control of the loss function rather than for the design of the loss function itself. For example, Wu et al. [162] proposed L2T-DLF to define the loss function of the model by another machine learning model and dynamically and automatically output the appropriate loss function to train the model during the training process. The teacher model dynamically regulates the student model process by outputting different loss functions according to the state of the student model at different stages of training. Such loss functions do not depend on specific tasks or optimization processes, while the training state of the model at each stage is more closely linked to the losses.

Wang et al. [23] argued that the combination of cross-entropy loss and metric learning loss, while treating them equally during training, does not fully utilize the discriminative power of neural networks and advocated that the system should first learn the appropriate feature representation and then classify the samples into the correct labels. Similarly, in Li et al. [163] on node classification of graph neural networks by coordinating classification loss and neighbor-based triplet loss, it is advocated to let the model first learn the appro-

priate feature representation and then generate high-quality samples to correctly optimize the classifier.

6. Discussion

In this section, we discuss how to choose the appropriate evaluation system for practical applications and the differences between different studies, as shown in Table 5. The evaluation system for curriculum learning consists of a difficulty evaluator and a training scheduler, a loss evaluator, and, first of all, different difficulty evaluators corresponding to different domain tasks. For example, for tasks such as object detection and image classification in computer vision, difficulty evaluation is required from the perspective of the number of labels [60], the number of object categories [66], the background [66], and the distribution density (clustering) in the feature space [93,94] of the image. Most of the existing studies on heuristic difficulty evaluators take one dimension [3,19] for difficulty discussion, and one can try to take a combination of multi-dimensional heuristic difficulty [164] metrics. Also for tasks in special domain contexts, such as medical image diagnosis, expert domain knowledge in such tasks is very important, and their sample difficulty is directly related to expert diagnostic opinions. Most difficulty evaluators for such tasks are based on expert annotations or image lesion degree [70,71].

Table 5. Selection recommendations for different evaluators.

Evaluator	Methods	Select Preferences
Difficulty evaluator	Heuristic	Multidimensional heuristic difficulty evaluators outperform single scales, and this type of evaluator is more suitable for tasks that rely on expert knowledge, such as in the medical domain.
	Non-heuristic	More suitable for most scenarios that are unfamiliar with datasets and do not depend on specific datasets and tasks.
Training scheduler	time	Root functions outperform other functions, and square root functions outperform the rest of the root functions.
		Dynamic model capability evaluators outperform static.
	weight	Dynamic thresholding outperforms thresholding with fixed parameters.
	proportion	Multiple metric thresholds are more comprehensive than single metric thresholds.
Loss evaluator		Evaluating the model's learning progress at each stage is better than not evaluating it.

In the context of practical applications, it is difficult and time-consuming to use the difficulty of defining samples from the data set structure or the problem itself. Moreover, the difficulty defined by humans and the difficulty learned by the model may not correspond. On the other hand, a predefined curriculum for a heuristic difficulty evaluator does not always apply to every training phase of the model because model training is a dynamic learning process. The choice of a non-heuristic difficulty evaluator is more closely related to the training model, and the non-heuristic difficulty evaluator does not require manual design and moderation, and the difficulty scores of the samples are directly obtained from the model or algorithm, such as model loss [25,29], degree of model improvement [103], and clustering [61]. A non-heuristic difficulty evaluator is more appropriate when we have no prior definition or are unfamiliar with the dataset.

For the training scheduler, the timing, weighting, and proportioning methods for the sample scheduling of interest in this paper are not completely separated. In most

of the literature, multiple strategies are adopted to be used together. For example, by dividing the samples into different fragments, the timing of adding fragments in each iteration is judged based on the model convergence [93]. Whether used individually or in combination, a training scheduler that is dynamically tuned during the model training process is preferred over a pre-fixed training scheduler. Because the pre-fixed training scheduler is used to manually estimate the learning progress of the model, the pre-set speed or time of adding new samples by this type of training scheduler may not match the current model capability, and the rate and time of adding samples cannot be properly controlled for the phase when the model is rapidly improving its capability or slowly improving. In general, we summarize the following points for the selection of the training scheduler: (i) The root function [3] outperforms the rest of the functions, and the square root function [91] outperforms the rest of the root functions in the velocity method using functions to control sample accessions. (ii) Using evaluation model capability scheduling, the dynamic model capability evaluator [25,97] outperforms the static model capability evaluator [3,144]. (iii) Focusing on the proportion of sample scheduling, dynamic thresholds [41,157] outperform thresholds with fixed parameters [12,32], and multiple metric thresholds [79,114] are more comprehensive than single metric thresholds [12,32]. Conventional curriculum learning (from easy to hard) such as Mixed [12] and Single [45] scheduling outperforms the Reverse Scheduling strategy [89,110]. For loss evaluator design, an approach that uses each stage to evaluate the model's learning progress [69,85] is superior to the approach without evaluation.

Regarding the three main directions of curriculum learning, self-paced learning, and anti-curriculum learning, primitive curriculum learning focuses more on the importance of prior knowledge, while self-paced learning emphasizes the loss in model training, and anti-curriculum learning, in contrast to curriculum learning, follows a hard-to-easy training sequence. For self-paced learning, schemes that embed the remaining regularizer [20,36] (e.g., the priori diversity regularizer) outperform schemes that do not embed the remaining regularizer [18]. Soft regularizer schemes [105] outperformed hard regularizer schemes. For various individual weighting schemes (mixed [20], linear [18], logarithmic [151], etc.), no single weighting scheme can be optimal for all datasets, and experimental and comparative analysis of the datasets is required.

Various variants derived today address the problems of curriculum learning focusing on prior knowledge and ignoring model progress and self-paced learning focusing on model progress and ignoring prior knowledge, such as self-paced curriculum learning (SPCL) [21], Collaborative Self-Paced Curriculum Learning (C-SPCL) [60], etc., or implicit curriculum learning that assigns a learnable variable to a sample as an important indicator [165]. These types of curriculum learning methods can effectively deal with prior knowledge and model progress, and they combine the ideas of primal curriculum learning and self-paced learning, which are somewhat superior to primal curriculum learning and self-paced learning. In addition, curriculum learning considers noise by effectively designing a curriculum that includes noise [93,158,166]. This type of curriculum learning method outperforms methods that discard noise completely or include noise directly for training.

Finally, we discuss the differences in terms mentioned in the review of curriculum learning directions and related studies. In the review by Wang et al. [167] and others, the general framework of curriculum learning is defined as "Difficulty Measurer + Training Scheduler", which divides curriculum learning into Predefined curriculum learning and Automatic curriculum learning. Wang et al. refer to both the Difficulty Measurer and the Training Scheduler as Predefined curriculum learning when they are designed based on human a priori knowledge and do not involve data-driven algorithms. Automatic curriculum learning is when one of them involves a data-driven model or algorithm. In this paper, curriculum learning is divided into three main parts: difficulty evaluator, training scheduler, and loss evaluator. The Predefined curriculum learning in Wang et al.'s study is similar to the heuristic difficulty evaluator (Section 2.1) and static training scheduler (Section 3.1) in this paper. Also, Wang et al. classify the difficulty evaluators in Predefined

curriculum learning as Discrete schedulers and Continuous schedulers, where the Discrete schedulers are similar to the study summarized in the fragmented training scheduler (Section 4.3.2) defined in this paper and the Continuous schedulers measure is similar to the static training scheduler (Section 4.1.1) controlled by the speed function in this paper. In the study by Soviany [168] et al., a data-level and model-level curriculum learning framework is defined, which consists of two elements: “curriculum scheduler and performance measure.” The curriculum scheduler is used to decide when to update the training subset, and the performance measure is used to evaluate the model’s performance, which is similar to the training scheduler and loss evaluator in this paper.

7. Machine Learning Concepts Similar to Curriculum Learning

Data selection strategies similar to curriculum learning methods exist in the field of machine learning, both of which use a certain plan to select training samples for the training process and dynamically sample small batches of samples. However, curriculum learning relies on the ranking of samples, and most of the sample rankings for curriculum learning are task-based, while these methods are based more on the current difficulties of the model in learning.

Active learning. Active learning focuses on the uncertainty of samples and achieves the expected performance of the model with as few labeled samples as possible by actively prioritizing the most valuable samples for labeling. The query strategy of active learning is similar to the design of the difficulty evaluator in the curriculum learning in this paper, which is also the core of active learning, and the uncertainty of concern includes density-weighted methods, expected model change, variance reduction, etc. The purpose of active learning is mainly to reduce the labeling cost and rapidly improve the model effect. In terms of query strategy and effect, active learning is very similar to curriculum learning, but by the starting point, active learning deals with unsupervised data, while curriculum learning methods are designed for all types of data, including supervised, unsupervised, weakly supervised, etc. Some studies combine active learning with self-paced learning, where the model considers both the difficulty and uncertainty of the samples during the training process [36].

Hard example mining (HEM). Hard Example Mining is also a widely researched data selection strategy. In contrast to curriculum learning, in each training cycle, HEM selects the hardest samples for training, and somehow suppresses a large number of easy negative examples to mine the information of all hard samples, which is used to solve the problems of sample imbalance and too many easy samples. Hard Example Mining includes Hard Negative Mining, Online Hard Example Mining (OHEM), etc. The difference is that Hard Negative Mining only focuses on hard negative examples, while OHEM focuses on all hard examples, regardless of positive and negative aspects. Hard data mining is more suitable for cleaner datasets, while curriculum learning is more suitable for data with more noise or outliers and may be preferable to hard data mining when the task is difficult [169].

Focal loss. A loss function, obtained by modifying the standard cross-entropy loss, makes the model focus more on hard-to-classify samples during training by reducing the weight of easy-to-classify samples, which is essentially a function for measuring the contribution of hard-to-classify and easy-to-classify samples to the total loss. Similar to the study of curriculum learning embedded in the loss function, which makes the model focus on some samples during training by a specific loss function, curriculum learning is used to guide the model to focus on easy to hard samples by the loss function.

Spaced repetition. A spaced repetition-based data sampling strategy mimics human learners and can learn more effectively by reviewing previously learned knowledge. It samples unlabeled data considering the difficulty of the sample and the ability of the model [170]. The spaced repetition strategy shares a similar difficulty evaluation and training scheduling approach to curriculum learning for determining when new samples should be added. The spaced repetition strategy uses Leitner queues for sample difficulty evaluation, initially placing all samples in the first queue and boosting samples with

correct predictions to one queue and samples with incorrect predictions to one queue lower, with higher queues accumulating samples that are easier for the model and lower queues accumulating samples that are more difficult as the prediction progresses. This approach, based on repeated sampling, has been similarly studied in the curriculum learning training scheduler. The difference consists in the interval repetition selection of all training samples already learned, and in the strengthening strategy for curriculum learning (Section 4.3.2) that selects difficult samples for repetitive learning used to enhance the model's generalization ability.

Boosting. Boosting is a method to turn weak classifiers into strong ones by using an initial training set to train a base classifier. The training sample distribution is adjusted according to the performance of the model by giving more weight to the previously misclassified samples, and the next base classifier is trained based on the adjusted sequence distribution until the base classifier reaches the target value, and finally by weighting these base classifiers together. In terms of focusing on sample categories, boosting focuses on misclassified samples, while curriculum learning focuses on samples of different difficulty categories at different times instead of focusing on only a single category of samples.

8. Case Study

In this section, we cite two classification works for a case study of curriculum learning. Wei et al. [95] used curriculum learning for optimization of colorectal polyp classification by evaluating the degree of agreement of expert annotators on each pathology image (Section 3.2.1) for sample difficulty and dividing the training into four stages, starting with easy images (i.e., samples in which all annotators agree perfectly on the pathology classification of that image) and gradually adding samples of difficult images with disagreement as training. The experimental results of this study outperformed all single-stage models in the second stage of training with an AUC of 85.5% and reached a maximum of 88.2% in the third stage, which is a 4.5% improvement compared to the baseline model. In addition, experimental results from the fourth stage of training showed that adding difficult images for training improved the classification performance of the model not only on difficult images but also on easy images.

Guo et al. [93] proposed the CurriculumNet framework for image classification tasks for efficiently handling the large amount of noisy data in a dataset. The researchers trained a dataset without any manual annotation using the Inception v2 model and obtained three subsets: clean, noisy, and high noisy, using a density-based clustering algorithm (Section 3.2.4), whose complexity gradually increases. The researchers divided the training into three stages (Section 3.1.2), with the first stage using only clean subsets for training and gradually adding subsets of data containing noisy and highly noisy data. Four comparison schemes are given in Guo et al.'s experiments, and the Top-1 and Top-5 results on both the WebVision and ImageNet datasets are shown in Table 6. Among them, the models proposed in the paper all outperform the comparison schemes and have better convergence speeds. In particular, the researchers explore the importance of high-noise subsets for model performance improvement and try to use 0–100% sampling experiments on high-noise subsets based on the original curriculum design, and the experimental results achieve optimal results for both Top-1 and Top-5 of the model at 50% of the high-noise subsets ratio. It shows that the third stage of learning for the high-noise samples is important for the model's performance improvement. This is similar to the conclusion of Wei et al. and demonstrates the importance of the difficulty relationship in the curriculum learning focus samples. In addition, the researchers compared the proposed method with the current state-of-the-art methods developed for learning labeled noisy samples in experimental trials, and the experimental results show that the proposed curriculum learning framework outperforms the state-of-the-art methods in both cases.

The above case study demonstrates the effectiveness of curriculum learning on the classification problem, which significantly improves the classification performance of the model through a reasonable difficulty evaluator and training scheduler design. The

effectiveness of curriculum learning in focusing on sample difficulty and the importance of difficult samples for model performance improvement are also demonstrated by separate experiments for difficult sample training.

Table 6. The experiments on the curriculum learning model (modified table from [93]).

Method	WebVision		Select Preferences	
	Top-1	Top-5	Top-5	Top-1
Using the whole dataset	30.16	12.43	36.00	16.20
Only using the clean subset	30.28	12.98	37.09	16.42
Using the proposed curriculum learning strategy on: clean and noisy subsets	28.44	11.38	35.66	15.24
Using the proposed curriculum learning strategy on: clean, noisy, and highly noisy subsets	27.91	10.82	35.24	15.11

9. Summary and Prospects

In the field of machine learning, optimization of deep learning models has become an important problem for various tasks. The curriculum learning approach focuses on the training order of samples in the model training process, guiding the model to the global optimum in an easy-to-hard order, so the curriculum learning method has attracted much attention from researchers in recent years. In this paper, we focus on the use of curriculum learning methods. First, we introduce the research history and basic concepts of curriculum learning, and then we introduce the existing objects to which curriculum learning is applied, including data-based curriculum learning, task-based curriculum learning, and model-based curriculum learning. After that, we describe the three major evaluators of curriculum learning methods, including the difficulty evaluator, the training scheduler, and the loss evaluator. Based on the results of the summary of the curriculum learning evaluation system, we propose several issues that need attention and research directions that deserve further investigation.

Low resource issues. Most applications of curriculum learning are used in optimization directions such as accelerating convergence and improving model performance. In the latest research experiments, it is proposed that curriculum learning outperforms ordinary training methods when the amount of data is limited, and the gap between curriculum learning and other methods gradually narrows as the amount of data gradually increases. Since this is the case, whether the curriculum learning method can be optimized by a reasonable curriculum design to achieve the full exploitation and utilization of a small amount of data, or combined with methods that deal with low-resource problems (such as meta-learning), is the key to dealing with low-resource problems through the curriculum learning method. For example, repetitive enhancement training for partial data, etc., and designing a reasonable curriculum arrangement is a challenging task.

Model learning hypothesis problem. The original assumption of curriculum learning was based on the human learning process, in which ability grows gradually with knowledge, but recently some researchers have pointed out that the assumption that models grow gradually in ability as they undergo training is incorrect. Intuitively, the ability of the model grows as the training sample size increases, but machine learning models constantly learn to fit new data, and parameters change constantly, which may then cause forgetting problems. There are few studies on the theoretical analysis of the effectiveness of curriculum learning, and more research is needed on how to make curriculum learning stable and effective on the task through theoretical analysis and design.

Noise Curriculum learning. Curriculum learning is currently less experimented in practical applications where noise corruption is a common problem and most of the datasets are costly to acquire and label. Learning in noisy data scenarios is an area worth exploring.

By using curriculum learning to effectively evaluate and schedule data sets containing noise, different noises (e.g., audio noise, labeled noise, etc.) are evaluated to achieve the desired performance of the model with the low-cost work of collecting samples and labeling them effectively.

Curriculum learning methods are still widely used at present. In this paper, we only summarize the existing methods in the fields of computer vision, natural language processing, medical diagnosis, network security, etc. Currently, different fields and tasks are also being gradually tapped into by researchers, such as system security in the field of cyberspace security, etc. We believe that curriculum learning will be applied in more fields.

Author Contributions: Conceptualization, T.Z. and C.Z.; Formal analysis, F.L.; Funding acquisition, L.W., L.L. and B.L. Writing—original draft, T.Z. and L.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the Hebei Province Professional Degree Teaching Case Establishment and Construction Project (Chunying Zhang: No. KCJSZ2022073), the Basic Scientific Research Business Expenses of Hebei Provincial Universities (Liya Wang: No. JST2022001), and the Tangshan Science and Technology Project (Liya Wang: No. 22130225G).

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Acknowledgments: Support by colleagues and the university is acknowledged.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Bengio, Y.; Louradour, J.; Collobert, R.; Weston, J. Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning, Montreal, QC, Canada, 14–18 June 2009; pp. 41–48.
2. Kumar, M.P.; Turki, H.; Preston, D.; Koller, D. Learning specific-class segmentation from diverse data. In Proceedings of the 2011 International Conference on Computer Vision, Barcelona, Spain, 6–13 November 2011; pp. 1800–1807.
3. Platanios, E.A.; Stretcu, O.; Neubig, G.; Poczos, B.; Mitchell, T.M. Competence-based curriculum learning for neural machine translation. *arXiv* **2019**, arXiv:1903.09848, 2019.
4. Manela, B.; Biess, A. Curriculum learning with hindsight experience replay for sequential object manipulation tasks. *Neural Netw.* **2022**, *145*, 260–270. [[CrossRef](#)] [[PubMed](#)]
5. Liu, N.; Lu, T.; Cai, Y.; Wang, S. Manipulation skill learning on multi-step complex task based on explicit and implicit curriculum learning. *Sci. China Inf. Sci.* **2022**, *65*, 114201. [[CrossRef](#)]
6. Zhao, R.; Chen, X.; Chen, Z.; Li, S. Diagnosing glaucoma on imbalanced data with self-ensemble dual-curriculum learning. *Med. Image Anal.* **2022**, *75*, 102295. [[CrossRef](#)]
7. Li, J.; Yang, B.; Yu, T. Distributed deep reinforcement learning-based coordination performance optimization method for proton exchange membrane fuel cell system. *Sustain. Energy Technol. Assess.* **2022**, *50*, 101814. [[CrossRef](#)]
8. Li, J.; Geng, J.; Yu, T. Grid-area coordinated load frequency control strategy using large-scale multi-agent deep reinforcement learning. *Energy Rep.* **2022**, *8*, 255–274. [[CrossRef](#)]
9. Chen, Y.; Wang, X.; Fan, M.; Huang, J.; Yang, S.; Zhu, W. Curriculum meta-learning for next POI recommendation. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual Event Singapore, 14–18 August 2021; pp. 2692–2702.
10. Liu, Z.; Cao, W.; Gao, Z.; Bian, J.; Chen, H.; Chang, Y.; Liu, T.-Y. Self-paced ensemble for highly imbalanced massive data classification. In Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE), Dallas, TX, USA, 20–24 April 2020; pp. 841–852.
11. Sarafianos, N.; Giannakopoulos, T.; Nikou, C.; Kakadiaris, I.A. Curriculum learning of visual attribute clusters for multi-task classification. *Pattern Recognit.* **2018**, *80*, 94–108. [[CrossRef](#)]
12. Xu, B.; Zhang, L.; Mao, Z.; Wang, Q.; Xie, H.; Zhang, Y. Curriculum learning for natural language understanding. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 6095–6104.
13. Skinner, B.F. *The Behavior of Organisms: An Experimental Analysis*; BF Skinner Foundation: Cambridge, MA, USA, 1938.
14. Krueger, K.A.; Dayan, P. Flexible shaping: How learning in small steps helps. *Cognition* **2009**, *110*, 380–394. [[CrossRef](#)]
15. Selfridge, O.G.; Sutton, R.S.; Barto, A.G. Training and Tracking in Robotics. *Ijcai* **1985**, 670–672.
16. Elman, J.L. Learning and development in neural networks: The importance of starting small. *Cognition* **1993**, *48*, 71–99. [[CrossRef](#)]
17. Allgower, E.L.; Georg, K. *Numerical Continuation Methods: An Introduction*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2012.

18. Kumar, M.; Packer, B.; Koller, D. Self-paced learning for latent variable models. In Proceedings of the 24th Annual Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 6–9 December 2010.
19. Spitkovsky, V.I.; Alshawi, H.; Jurafsky, D. From baby steps to leapfrog: How “less is more” in unsupervised dependency parsing. In Proceedings of the Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics, Los Angeles, CA, USA, 2–4 June 2010; pp. 751–759.
20. Jiang, L.; Meng, D.; Yu, S.I.; Lan, Z.; Shan, S.; Hauptmann, A. Self-paced learning with diversity. *Adv. Neural Inf. Process. Syst.* **2014**, *27*, 2078–2086.
21. Jiang, L.; Meng, D.; Zhao, Q.; Shan, S.; Hauptmann, A.G. Self-paced curriculum learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Austin, TX, USA, 25–30 January 2015; Volume 29.
22. Li, H.; Gong, M.; Meng, D.; Miao, Q. Multi-objective self-paced learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Phoenix, AZ, USA, 12–17 February 2016; Volume 30.
23. Wang, Y.; Gan, W.; Yang, J.; Wu, W.; Yan, J. Dynamic curriculum learning for imbalanced data classification. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 5017–5026.
24. Zhao, R.; Chen, X.; Chen, Z.; Li, S. EGDCL: An adaptive curriculum learning framework for unbiased glaucoma diagnosis. In Proceedings of the Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, 23–28 August 2020; Part 16; Springer International Publishing: Berlin/Heidelberg, Germany, 2020; pp. 190–205.
25. Gan, Z.; Xu, H.; Zan, H. Self-supervised curriculum learning for spelling error correction. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online, 7–11 November 2021; pp. 3487–3494.
26. Dai, Y.; Li, H.; Li, Y.; Sun, J.; Huang, F.; Si, L.; Zhu, X. Preview, attend and review: Schema-aware curriculum learning for multi-domain dialog state tracking. *arXiv* **2021**, arXiv:2106.00291.
27. Li, S.; Yang, B.; Zou, Y. Adaptive curriculum learning for video captioning. *IEEE Access* **2022**, *10*, 31751–31759. [[CrossRef](#)]
28. Matiisen, T.; Oliver, A.; Cohen, T.; Schulman, J. Teacher–student curriculum learning. *IEEE Trans. Neural Netw. Learn. Syst.* **2019**, *31*, 3732–3740. [[CrossRef](#)] [[PubMed](#)]
29. Kesgin, H.T.; Amasyali, M.F. Cyclical Curriculum Learning. *arXiv* **2022**, arXiv:2202.05531.
30. Yang, Z.Y.; Xia, L.Y.; Zhang, H.; Liang, Y. MSPL: Multimodal self-paced learning for multi-omics feature selection and data integration. *IEEE Access* **2019**, *7*, 170513–170524. [[CrossRef](#)]
31. Zhang, D.; Meng, D.; Han, J. Co-saliency detection via a self-paced multiple-instance learning framework. *IEEE Trans. Pattern Anal. Mach. Intell.* **2016**, *39*, 865–878. [[CrossRef](#)]
32. Shen, L.; Feng, Y. CDL: Curriculum dual learning for emotion-controllable response generation. *arXiv* **2020**, arXiv:2005.00329.
33. Zhang, W.; Geng, S.; Fu, Z.; Zheng, L.; Jiang, C.; Hong, S. MetaVA: Curriculum Meta-learning and Pre-fine-tuning of Deep Neural Networks for Detecting Ventricular Arrhythmias based on ECGs. *arXiv* **2022**, arXiv:2202.12450.
34. Morerio, P.; Cavazza, J.; Volpi, R.; Vidal, R.; Murino, V. Curriculum dropout. In Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 3544–3552.
35. Dong, Q.; Gong, S.; Zhu, X. Multi-task curriculum transfer deep learning of clothing attributes. In Proceedings of the 2017 IEEE Winter Conference on Applications of Computer Vision (WACV), Sanat Rosa, CA, USA, 24–31 March 2017; pp. 520–529.
36. Tang, Y.P.; Huang, S.J. Self-paced active learning: Query the right thing at the right time. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 29–31 January 2019; Volume 33, pp. 5117–5124.
37. Ge, Y.; Zhu, F.; Chen, D.; Zhao, R. Self-paced contrastive learning with hybrid memory for domain adaptive object re-id. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 11309–11321.
38. Pi, T.; Li, X.; Zhang, Z.; Wu, F.; Xiao, J.; Zhuang, Y. Self-paced boost learning for classification. In Proceedings of the 25th International Joint Conference on Artificial Intelligence IJCAI-16, New York, NY, USA, 9–13 July 2016; pp. 1932–1938.
39. Liu, F.; Tian, Y.; Chen, Y.; Liu, Y. ACPL: Anti-curriculum pseudo-labelling for semi-supervised medical image classification. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, New Orleans, LA, USA, 19–20 June 2022; pp. 20697–20706.
40. Cascante-Bonilla, P.; Tan, F.; Qi, Y.; Ordonez, V. Curriculum labeling: Revisiting pseudo-labeling for semi-supervised learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Online, 2–9 February 2021; Volume 35, pp. 6912–6920.
41. Zhang, B.; Wang, Y.; Hou, W.; Wang, J.; Okumura, M.; Shinozaki, T. Flexmatch: Boosting semi-supervised learning with curriculum pseudo labeling. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 18408–18419.
42. Shu, Y.; Cao, Z.; Long, M.; Wang, J. Transferable curriculum for weakly-supervised domain adaptation. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 29–31 January 2019; Volume 33, pp. 4951–4958.
43. Ma, F.; Meng, D.; Xie, Q.; Li, Z.; Dong, X. Self-paced co-training. In Proceedings of the International Conference on Machine Learning, PMLR, Sydney, Australia, 6–11 August 2017; pp. 2275–2284.
44. Zhan, R.; Liu, X.; Wong, D.F.; Chao, L.S. Meta-curriculum learning for domain adaptation in neural machine translation. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2021; Volume 35, pp. 14310–14318.
45. Cirik, V.; Hovy, E.; Morency, L.P. Visualizing and understanding curriculum learning for long short-term memory networks. *arXiv* **2016**, arXiv:1611.06204.
46. Pentina, A.; Sharmanska, V.; Lampert, C.H. Curriculum learning of multiple tasks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston, MA, USA, 7–12 June 2015; pp. 5492–5500.

47. De Buysscher, D.; Pollack, T.; Van Kampen, E.J. Safe Curriculum Learning for Linear Systems with Parametric Unknowns in Primary Flight Control. In Proceedings of the AIAA SCITECH 2022 Forum, San Diego, CA, USA, 3–7 January 2022.
48. Mao, J.; Gan, C.; Kohli, P.; Tenenbaum, J.B.; Wu, J. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. *arXiv* **2019**, arXiv:1904.12584.
49. Ma, H.; Dong, D.; Ding, S.X.; Chen, C. Curriculum-based deep reinforcement learning for quantum control. *IEEE Trans. Neural Netw. Learn. Syst.* **2022**, 1–14. [\[CrossRef\]](#)
50. Zhang, Y.; David, P.; Foroosh, H.; Gong, B. A curriculum domain adaptation approach to the semantic segmentation of urban scenes. *IEEE Trans. Pattern Anal. Mach. Intell.* **2019**, 42, 1823–1841. [\[CrossRef\]](#)
51. Sarafianos, N.; Giannakopoulos, T.; Nikou, C.; Kakadiaris, I.A. Curriculum learning for multi-task classification of visual attributes. In Proceedings of the IEEE International Conference on Computer Vision Workshops, Venice, Italy, 22–29 October 2017; pp. 2608–2615.
52. Fu, P.; Zhang, D.; Yin, F.; Tang, H. The multi-mode operation decision of cleaning robot based on curriculum learning strategy and feedback network. *Neural Comput. Appl.* **2022**, 34, 9955–9966. [\[CrossRef\]](#)
53. Karras, T.; Aila, T.; Laine, S.; Lehtinen, J. Progressive growing of gans for improved quality, stability, and variation. *arXiv* **2017**, arXiv:1710.10196.
54. Ayyubi, H.A.; Yao, Y.; Divakaran, A. Progressive Growing of Neural ODEs. *arXiv* **2020**, arXiv:2003.03695.
55. Sinha, S.; Garg, A.; Laroche, H. Curriculum by smoothing. *Adv. Neural Inf. Process. Syst.* **2020**, 33, 21653–21664.
56. Kurmi, V.K.; Bajaj, V.; Subramanian, V.K.; Nambodiri, V.P. Curriculum based dropout discriminator for domain adaptation. *arXiv* **2019**, arXiv:1907.10628.
57. Sharma, R.; Barratt, S.; Ermon, S.; Pande, V. Improved training with curriculum gans. *arXiv* **2018**, arXiv:1807.09295.
58. Doan, T.; Monteiro, J.; Albuquerque, I.; Mazouze, B.; Durand, A.; Pineau, J.; Hjelm, R.D. On-line adaptive curriculum learning for gans. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 29–31 January 2019; Volume 33, pp. 3470–3477.
59. Lee, Y.J.; Grauman, K. Learning the easy things first: Self-paced visual category discovery. In Proceedings of the CVPR 2011, Washington, DC, USA, 20–25 June 2011; pp. 1721–1728.
60. Zhang, D.; Han, J.; Zhao, L.; Meng, D. Leveraging prior-knowledge for weakly supervised object detection under a collaborative self-paced curriculum learning framework. *Int. J. Comput. Vis.* **2019**, 127, 363–380. [\[CrossRef\]](#)
61. Oksuz, I.; Ruijsink, B.; Puyol-Antón, E.; Clough, J.R.; Cruz, G.; Bustin, A.; Prieto, C.; Botnar, R.; Rueckert, D.; Schnabel, J.A.; et al. Automatic CNN-based detection of cardiac MR motion artefacts using k-space data augmentation and curriculum learning. *Med. Image Anal.* **2019**, 55, 136–147. [\[CrossRef\]](#)
62. Pan, Y.; Li, Z.; Zhang, L.; Tang, J. Causal Inference with Knowledge Distilling and Curriculum Learning for Unbiased VQA. *7ACM Trans. Multimed. Comput. Commun. Appl. (TOMM)* **2022**, 18, 67. [\[CrossRef\]](#)
63. Li, Z.; Yang, J.; Liu, Z.; Yang, X.; Jeon, G.; Wu, W. Feedback network for image super-resolution. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 3867–3876.
64. Wang, Y.; Perazzi, F.; McWilliams, B.; Sorkine-Hornung, A.; Sorkine-Hornung, O.; Schroers, C. A fully progressive approach to single-image super-resolution. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, Salt Lake City, UT, USA, 18–22 June 2018; pp. 864–873.
65. Tudor Ionescu, R.; Alexe, B.; Leordeanu, M.; Popescu, M.; Papadopoulos, D.P.; Ferrari, V. How hard can it be? Estimating the difficulty of visual search in an image. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 26 June–1 July 2016; pp. 2157–2166.
66. Wei, Y.; Liang, X.; Chen, Y.; Shen, X.; Cheng, M.-M.; Feng, J.; Zhao, Y.; Yan, S. Stc: A simple to complex framework for weakly-supervised semantic segmentation. *IEEE Trans. Pattern Anal. Mach. Intell.* **2016**, 39, 2314–2320. [\[CrossRef\]](#)
67. Gui, L.; Baltrušaitis, T.; Morency, L.P. Curriculum learning for facial expression recognition. In Proceedings of the 2017 12th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2017), Washington, DC, USA, 30 May–3 June 2017; pp. 505–511.
68. Zhu, J.; Li, D.; Han, T.; Tian, L.; Shan, Y. Progressface: Scale-aware progressive learning for face detection. In Proceedings of the Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, 23–28 August 2020; Part 6; Springer International Publishing: Berlin/Heidelberg, Germany, 2020; pp. 344–360.
69. Gao, R.; Grauman, K. On-demand learning for deep image restoration. In Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 1086–1095.
70. Tang, Y.; Wang, X.; Harrison, A.P.; Lu, L.; Xiao, J.; Summers, R.M. Attention-guided curriculum learning for weakly supervised classification and localization of thoracic diseases on chest radiographs. In Proceedings of the Machine Learning in Medical Imaging: 9th International Workshop, MLMI 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, 16 September 2018; Springer International Publishing: Berlin/Heidelberg, Germany, 2018; pp. 249–258.
71. Jesson, A.; Guizard, N.; Ghalehjegh, S.H.; Goblot, D.; Soudan, F.; Chapados, N. CASED: Curriculum adaptive sampling for extreme data imbalance. In Proceedings of the Medical Image Computing and Computer Assisted Intervention—MICCAI 2017: 20th International Conference, Quebec City, QC, Canada, 11–13 September 2017; Part III. Springer International Publishing: Berlin/Heidelberg, Germany, 2017; pp. 639–646.

72. Zhang, Y.; David, P.; Gong, B. Curriculum domain adaptation for semantic segmentation of urban scenes. In Proceedings of the IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 2020–2030.
73. Rajeswar, S.; Subramanian, S.; Dutil, F.; Pal, C.; Courville, A. Adversarial generation of natural language. *arXiv* **2017**, arXiv:1705.10929.
74. Yu, Y.; Zhang, W.; Hasan, K.; Yu, M.; Xiang, B.; Zhou, B. End-to-end answer chunk extraction and ranking for reading comprehension. *arXiv* **2016**, arXiv:1610.09996.
75. Tay, Y.; Wang, S.; Tuan, L.A.; Fu, J.; Phan, M.C.; Yuan, X.; Rao, J.; Hui, S.C.; Zhang, A. Simple and effective curriculum pointer-generator networks for reading comprehension over long narratives. *arXiv* **2019**, arXiv:1905.10847.
76. Kocmi, T.; Bojar, O. Curriculum learning and minibatch bucketing in neural machine translation. *arXiv* **2017**, arXiv:1707.09533.
77. Tsvetkov, Y.; Faruqui, M.; Ling, W.; MacWhinney, B.; Dyer, C. Learning the curriculum with bayesian optimization for task-specific word representation learning. *arXiv* **2016**, arXiv:1605.03852.
78. Press, O.; Bar, A.; Bogin, B.; Berant, J.; Wolf, L. Language generation with recurrent generative adversarial networks without pre-training. *arXiv* **2017**, arXiv:1706.01399.
79. Liu, F.; Ge, S.; Wu, X. Competence-based multimodal curriculum learning for medical report generation. *arXiv* **2022**, arXiv:2206.14579.
80. Ruiter, D.; van Genabith, J.; España-Bonet, C. Self-induced curriculum learning in self-supervised neural machine translation. *arXiv* **2020**, arXiv:2004.03151.
81. Wang, W.; Tian, Y.; Ngiam, J.; Yang, Y.; Caswell, I.; Parekh, Z. Learning a multi-domain curriculum for neural machine translation. *arXiv* **2019**, arXiv:1908.10940.
82. Lu, Y.; Lin, H.; Xu, J.; Han, X.; Tang, J.; Li, A.; Sun, L.; Liao, M.; Chen, S. Text2event: Controllable sequence-to-structure generation for end-to-end event extraction. *arXiv* **2021**, arXiv:2106.09232.
83. Wang, C.; Wu, Y.; Liu, S.; Zhou, M.; Yang, Z. Curriculum pre-training for end-to-end speech translation. *arXiv* **2020**, arXiv:2004.10093.
84. Ranjan, S.; Hansen, J.H.L. Curriculum learning based approaches for noise robust speaker recognition. *IEEE/ACM Trans. Audio Speech Lang. Process.* **2017**, *26*, 197–210. [[CrossRef](#)]
85. Ng, D.; Chen, Y.; Tian, B.; Fu, Q.; Chng, E.S. Convmixer: Feature interactive convolution with curriculum learning for small footprint and noisy far-field keyword spotting. In Proceedings of the ICASSP 2022–2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Singapore, 22–27 May 2022; pp. 3603–3607.
86. Braun, S.; Neil, D.; Liu, S.C. A curriculum learning method for improved noise robustness in automatic speech recognition. In Proceedings of the 2017 25th European Signal Processing Conference (EUSIPCO), Kos, Greece, 28 August–2 September 2017; pp. 548–552.
87. Amodei, D.; Ananthanarayanan, S.; Anubhai, R.; Bai, J.; Battenberg, E.; Case, C.; Casper, J.; Catanzaro, B.; Cheng, Q.; Chen, G.; et al. Deep speech 2: End-to-end speech recognition in English and mandarin. In Proceedings of the International Conference on Machine Learning, PMLR, New York, NY, USA, 20–22 June 2016; pp. 173–182.
88. Takahashi, N.; Singh, M.K.; Mitsufuji, Y. Source Mixing and Separation Robust Audio Steganography. *arXiv* **2021**, arXiv:2110.05054.
89. Zhang, X.; Kumar, G.; Khayrallah, H.; Murray, K.; Gwinnup, J.; Martindale, M.J.; McNamee, P.; Duh, K.; Carpuat, M. An empirical exploration of curriculum learning for neural machine translation. *arXiv* **2018**, arXiv:1811.00739.
90. Hachohen, G.; Weinshall, D. On the power of curriculum learning in training deep networks. In Proceedings of the International Conference on Machine Learning, PMLR, Vancouver, BC, Canada, 13 October 2019; pp. 2535–2544.
91. Penha, G.; Hauff, C. Curriculum learning strategies for ir: An empirical study on conversation response ranking. In Proceedings of the Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, 14–17 April 2020; Part I. Springer International Publishing: Berlin/Heidelberg, Germany, 2020; pp. 699–713.
92. Weinshall, D.; Cohen, G.; Amir, D. Curriculum learning by transfer learning: Theory and experiments with deep networks. In Proceedings of the International Conference on Machine Learning, PMLR, Beijing, China, 14–16 November 2018; pp. 5238–5246.
93. Guo, S.; Huang, W.; Zhang, H.; Zhuang, C.; Dong, D.; Scott, M.R.; Huang, D. Curriculumnet: Weakly supervised learning from large-scale web images. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 135–150.
94. Liu, X.; Zhou, F.; Shen, D.; Wang, S. Deep convolutional neural networks with curriculum learning for facial expression recognition. In Proceedings of the 2019 Chinese Control and Decision Conference (CCDC), Nanchang, China, 3–5 June 2019; pp. 5925–5932.
95. Wei, J.; Suriawinata, A.; Ren, B.; Liu, X.; Lisovsky, M.; Vaickus, L.; Brown, C.; Baker, M.; Nasir-Moin, M.; Tomita, N.; et al. Learn like a pathologist: Curriculum learning by annotator agreement for histopathology image classification. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, Waikoloa, HI, USA, 3–8 January 2021; pp. 2473–2483.
96. Jiménez-Sánchez, A.; Mateus, D.; Kirchhoff, S.; Kirchhoff, C.; Biberthaler, P.; Navab, N.; Ballester, M.A.G.; Piella, G. Curriculum learning for improved femur fracture classification: Scheduling data with prior knowledge and uncertainty. *Med. Image Anal.* **2022**, *75*, 102273. [[CrossRef](#)]
97. Lalor, J.P.; Yu, H. Dynamic data selection for curriculum learning via ability estimation. In Proceedings of the Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020; Volume 545.
98. Zhao, M.; Wu, H.; Niu, D.; Wang, X. Reinforced curriculum learning on pre-trained neural machine translation models. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 9652–9659.

99. Qu, M.; Tang, J.; Han, J. Curriculum learning for heterogeneous star network embedding via deep reinforcement learning. In Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, Marina Del Rey, CA, USA, 5–9 February 2018; pp. 468–476.
100. Narvekar, S.; Peng, B.; Leonetti, M.; Sinapov, J.; Taylor, M.E.; Stone, P. Curriculum learning for reinforcement learning domains: A framework and survey. *J. Mach. Learn. Res.* **2020**, *21*, 7382–7431.
101. Gupta, K.; Mukherjee, D.; Najjaran, H. Extending the capabilities of reinforcement learning through curriculum: A review of methods and applications. *SN Comput. Sci.* **2022**, *3*, 28. [\[CrossRef\]](#)
102. Kumar, G.; Foster, G.; Cherry, C.; Krikun, M. Reinforcement learning based curriculum optimization for neural machine translation. *arXiv* **2019**, arXiv:1903.00041.
103. Sachan, M.; Xing, E. Self-training for jointly learning to ask and answer questions. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Long Papers); Association for Computational Linguistics: New Orleans, LA, USA, 2018; Volume 1, pp. 629–640.
104. Xiang, L.; Ding, G.; Han, J. Learning from multiple experts: Self-paced knowledge distillation for long-tailed classification. In *Computer Vision, Proceedings of the ECCV 2020: 16th European Conference, Glasgow, UK, 23–28 August 2020, Proceedings, Part V 16*; Springer International Publishing: Berlin/Heidelberg, Germany, 2020; pp. 247–263.
105. Zhao, Q.; Meng, D.; Jiang, L.; Xie, Q.; Xu, Z.; Hauptmann, A. Self-paced learning for matrix factorization. In Proceedings of the AAAI Conference on Artificial Intelligence, Austin, TX, USA, 25–30 January 2015; Volume 29.
106. Spitzkovsky, V.I.; Alshaw, H.; Jurafsky, D. *Baby Steps: How “Less is More” in Unsupervised Dependency Parsing*; Association for Computational Linguistics: Los Angeles, CA, USA, 2009.
107. Zhou, T.; Wang, S.; Bilmes, J. Curriculum learning by dynamic instance hardness. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 8602–8613.
108. Durand, T.; Mehra, N.; Mori, G. Learning a deep convnet for multi-label classification with partial labels. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 647–657.
109. Kesgin, H.T.; Amasyali, M.F. Development and Comparison of Scoring Functions in Curriculum Learning. In Proceedings of the 2022 2nd International Conference on Computing and Machine Intelligence (ICMI), Istanbul, Turkey, 15–16 July 2022; pp. 1–6.
110. Zhang, X.; Shapiro, P.; Kumar, G.; McNamee, P.; Carpuat, M.; Duh, K. Curriculum learning for domain adaptation in neural machine translation. *arXiv* **2019**, arXiv:1905.05816.
111. Guo, M.; Haque, A.; Huang, D.A.; Yeung, S.; Fei-Fei, L. Dynamic task prioritization for multitask learning. In Proceedings of the European conference on computer vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 270–287.
112. Fan, Y.; He, R.; Liang, J.; Hu, B. Self-paced learning: An implicit regularization perspective. In Proceedings of the AAAI Conference on Artificial Intelligence, San Francisco, CA, USA, 6–9 February 2017; Volume 31.
113. Zhou, Y.; Yang, B.; Wong, D.F.; Wan, Y.; Chao, L.S. Uncertainty-aware curriculum learning for neural machine translation. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 6934–6944.
114. Dou, Z.Y.; Anastasopoulos, A.; Neubig, G. Dynamic data selection and weighting for iterative back-translation. *arXiv* **2020**, arXiv:2004.03672.
115. Wang, W.; Caswell, I.; Chelba, C. Dynamically composing domain-data selection with clean-data selection by “co-curricular learning” for neural machine translation. *arXiv* **2019**, arXiv:1906.01130.
116. Mousavi, H.; Imani, M.; Ghassemian, H. Deep curriculum learning for polar image classification. In Proceedings of the 2022 International Conference on Machine Vision and Image Processing (MVIP), Ahvaz, Iran, 23–24 February 2022; pp. 1–5.
117. Zhou, T.; Wang, S.; Bilmes, J. Robust curriculum learning: From clean label detection to noisy label self-correction. In Proceedings of the International Conference on Learning Representations, Online, 3–7 May 2021.
118. Guo, J.; Tan, X.; Xu, L.; Qin, T.; Chen, E.; Liu, T.-Y. Fine-tuning by curriculum learning for non-autoregressive neural machine translation. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 7839–7846.
119. Xia, Z.; Zhou, Y.; Yan, F.Y.; Jiang, J. Automatic Curriculum Generation for Learning Adaptation in Networking. *arXiv* **2022**, arXiv:2202.05940.
120. Maicas, G.; Bradley, A.P.; Nascimento, J.C.; Reid, I.; Carneiro, G. Training medical image analysis systems like radiologists. In *Medical Image Computing and Computer Assisted Intervention, Proceedings of the MICCAI 2018: 21st International Conference, Granada, Spain, 16–20 September 2018, Proceedings, Part I*; Springer International Publishing: Cham, Switzerland, 2018; pp. 546–554.
121. Jin, X.; Peng, B.; Wu, Y.; Liu, Y.; Liu, J.; Liang, D.; Yan, J.; Hu, X. Knowledge distillation via route constrained optimization. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Republic of Korea, 27 October–2 November 2019; pp. 1345–1354.
122. Liu, J.; Chen, Y.; Liu, H.; Zhang, H.; Zhang, Y. From Less to More: Progressive Generalized Zero-Shot Detection with Curriculum Learning. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 19016–19029. [\[CrossRef\]](#)
123. Huang, Y.; Du, J. Self-attention enhanced CNNs and collaborative curriculum learning for distantly supervised relation extraction. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, 3–7 November 2019; Association for Computational Linguistics: Hong Kong, China, 2019; pp. 389–398.

124. Soviany, P.; Ardei, C.; Ionescu, R.T.; Leordeanu, M. Image difficulty curriculum for generative adversarial networks (CuGAN). In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, Snowmass Village, CO, USA, 1–5 March 2020; pp. 3463–3472.
125. Shelke, O.; Meisheri, H.; Khadilkar, H. Identifying efficient curricula for reinforcement learning in complex environments with a fixed computational budget. In Proceedings of the 5th Joint International Conference on Data Science & Management of Data (9th ACM IKDD CODS and 27th COMAD), Bangalore, India, 8–10 January 2022; pp. 81–89.
126. Pang, Z.-J.; Liu, R.-Z.; Meng, Z.-Y.; Zhang, Y.; Yu, Y.; Lu, T. On reinforcement learning for full-length game of starcraft. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 26 January 2019; Volume 33, pp. 4691–4698.
127. Wu, Y.; Tian, Y. Training agent for first-person shooter game with actor-critic curriculum learning. In Proceedings of the International Conference on Learning Representations, Toulon, France, 24–26 April 2017.
128. Zhang, X.; Eseye, A.T.; Knueven, B.; Liu, W.; Reynolds, M.; Jones, W. Curriculum-based Reinforcement Learning for Distribution System Critical Load Restoration. *IEEE Trans. Power Syst.* 2022, early access. [[CrossRef](#)]
129. Choi, J.; Jeong, M.; Kim, T.; Kim, C. Pseudo-labeling curriculum for unsupervised domain adaptation. *arXiv* **2019**, arXiv:1908.00262.
130. Liu, Z.; Manh, V.; Yang, X.; Huang, X.; Lekadir, K.; Campello, V.; Ravikumar, N.; Frangi, A.F.; Ni, D. Style curriculum learning for robust medical image segmentation. In *Medical Image Computing and Computer Assisted Intervention, Proceedings of the MICCAI 2021: 24th International Conference, Strasbourg, France, 27 September–1 October 2021, Proceedings, Part I 24*; Springer International Publishing: Berlin/Heidelberg, Germany, 2021; pp. 451–460.
131. Graves, A.; Bellemare, M.G.; Menick, J.; Munos, R.; Kavukcuoglu, K. Automated curriculum learning for neural networks. In Proceedings of the International Conference on Machine Learning, PMLR, Sydney, NSW, Australia, 6–11 August 2017; pp. 1311–1320.
132. Huang, Y.; Wang, Y.; Tai, Y.; Liu, X.; Shen, P.; Li, S.; Li, J.; Huang, F. Curricularface: Adaptive curriculum learning loss for deep face recognition. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020; pp. 5901–5910.
133. Ghasedi, K.; Wang, X.; Deng, C.; Huang, H. Balanced self-paced learning for generative adversarial clustering network. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 4391–4400.
134. Luo, B.; Feng, Y.; Wang, Z.; Zhu, Z.; Huang, S.; Yan, R.; Zhao, D. Learning with noise: Enhance distantly supervised relation extraction with dynamic transition matrix. *arXiv* **2017**, arXiv:1705.03995.
135. Wu, X.; Dyer, E.; Neyshabur, B. When do curricula work? *arXiv* **2020**, arXiv:2012.03107.
136. Jiménez-Sánchez, A.; Mateus, D.; Kirchhoff, S.; Kirchhoff, C.; Biberthaler, P.; Navab, N.; Ballester, M.A.G.; Piella, G. Medical-based deep curriculum learning for improved fracture classification. In *Medical Image Computing and Computer Assisted Intervention. Proceedings of the MICCAI 2019: 22nd International Conference, Shenzhen, China, 13–17 October 2019, Proceedings, Part VI 22*; Springer International Publishing: Berlin/Heidelberg, Germany, 2019; pp. 694–702.
137. Haarbuerger, C.; Baumgartner, M.; Truhn, D.; Broeckmann, M.; Schneider, H.; Schrading, S.; Kuhl, C.; Merhof, D. Multi scale curriculum CNN for context-aware breast MRI malignancy classification. In *Medical Image Computing and Computer Assisted Intervention, Proceedings of the MICCAI 2019: 22nd International Conference, Shenzhen, China, 13–17 October 2019, Proceedings, Part IV 22*; Springer International Publishing: Berlin/Heidelberg, Germany, 2019; pp. 495–503.
138. Chung, J.S.; Zisserman, A.P. Lip reading in profile. In Proceedings of the BMVC 2017, London, UK, 4–7 September 2017.
139. Liu, Y.; Shi, M.; Zhao, Q.; Wang, X. Point in, box out: Beyond counting persons in crowds. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 6469–6478.
140. Chen, X.; Gupta, A. Webly supervised learning of convolutional networks. In Proceedings of the IEEE international conference on computer vision, Santiago, Chile, 7–13 December 2015; pp. 1431–1439.
141. Boroumand, M.; Chen, M.; Fridrich, J. Deep residual network for steganalysis of digital images. *IEEE Trans. Inf. Forensics Secur.* **2018**, *14*, 1181–1193. [[CrossRef](#)]
142. Ye, J.; Ni, J.; Yi, Y. Deep learning hierarchical representations for image steganalysis. *IEEE Trans. Inf. Forensics Secur.* **2017**, *12*, 2545–2557. [[CrossRef](#)]
143. Liu, X.; Lai, H.; Wong, D.F.; Chao, L.S. Norm-based curriculum learning for neural machine translation. *arXiv* **2020**, arXiv:2006.02014.
144. Agrawal, S.; Carpuat, M. An Imitation Learning Curriculum for Text Editing with Non-Autoregressive Models. *arXiv* **2022**, arXiv:2203.09486.
145. Liu, C.; He, S.; Liu, K.; Zhao, J. Curriculum Learning for Natural Answer Generation. In Proceedings of the IJCAI, Stockholm, Sweden, 13–19 July 2018; pp. 4223–4229.
146. Jiang, L.; Zhou, Z.; Leung, T.; Fei-Fei, L. Mentornet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In Proceedings of the International Conference on Machine Learning, PMLR, Stockholm, Sweden, 10–15 July 2018; pp. 2304–2313.
147. Luo, J.; Kitamura, G.; Doganay, E.; Arefan, D.; Wu, S. Medical knowledge-guided deep curriculum learning for elbow fracture diagnosis from x-ray images. In Proceedings of the Medical Imaging 2021: Computer-Aided Diagnosis, Online, 15–19 February 2021; Volume 11597, pp. 247–252.

148. Luo, J.; Kitamura, G.; Arefan, D.; Doganay, E.; Panigrahy, A.; Wu, S. Knowledge-guided multiview deep curriculum learning for elbow fracture classification. In *Machine Learning in Medical Imaging, Proceedings of the 12th International Workshop, MLMI 2021, Held in Conjunction with MICCAI 2021, Strasbourg, France, 27 September 2021, Proceedings 12*; Springer International Publishing: Berlin/Heidelberg, Germany, 2021; pp. 555–564.
149. Gao, H.; Huang, H. Self-paced network embedding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018*; pp. 1406–1415.
150. Xu, W.; Liu, W.; Chi, H.; Qiu, S.; Jin, Y. Self-paced learning with privileged information. *Neurocomputing* **2019**, *362*, 147–155. [\[CrossRef\]](#)
151. Jiang, L.; Meng, D.; Mitamura, T.; Hauptmann, A.G. Easy samples first: Self-paced reranking for zero-example multimedia search. In *Proceedings of the 22nd ACM international conference on Multimedia, Orlando, FL, USA, 3–7 November 2014*; pp. 547–556.
152. Li, C.; Yan, J.; Wei, F.; Dong, W.; Liu, Q.; Zha, H. Self-paced multi-task learning. In *Proceedings of the AAAI Conference on Artificial Intelligence, San Francisco, CA, USA, 4–9 February 2017*; Volume 31.
153. Gong, M.; Li, H.; Meng, D.; Miao, Q.; Liu, J. Decomposition-based evolutionary multiobjective optimization to self-paced learning. *IEEE Trans. Evol. Comput.* **2018**, *23*, 288–302. [\[CrossRef\]](#)
154. Han, L.; Zhang, D.; Huang, D.; Chang, X.; Ren, J.; Luo, S.; Han, J. Self-paced Mixture of Regressions. In *Proceedings of the IJCAI 2017, Melbourne, Australia, 19–25 August 2017*; pp. 1816–1822.
155. Li, C.; Wei, F.; Yan, J.; Zhang, X.; Liu, Q.; Zha, H. A self-paced regularization framework for multilabel learning. *IEEE Trans. Neural Netw. Learn. Syst.* **2017**, *29*, 2660–2666. [\[CrossRef\]](#)
156. Zhou, D.; He, J.; Yang, H.; Fan, W. Sparc: Self-paced network representation for few-shot rare category characterization. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, London, UK, 19–23 August 2018*; pp. 2807–2816.
157. Wang, C.; Jin, S.; Guan, Y.; Liu, W.; Qian, C.; Luo, P.; Ouyang, W. Pseudo-labeled auto-curriculum learning for semi-supervised keypoint localization. *arXiv* **2022**, arXiv:2201.08613.
158. Florensa, C.; Held, D.; Wulfmeier, M.; Zhang, M.; Abbeel, P. Reverse curriculum generation for reinforcement learning. In *Proceedings of the Conference on Robot Learning, PMLR, Seoul, Republic of Korea, 15–17 November 2017*; pp. 482–495.
159. Havaei, M.; Guizard, N.; Chapados, N.; Bengio, Y. Hemis: Hetero-modal image segmentation. In *Medical Image Computing and Computer-Assisted Intervention, Proceedings of the MICCAI 2016: 19th International Conference, Athens, Greece, 17–21 October 2016, Proceedings, Part II 19*; Springer International Publishing: Berlin/Heidelberg, Germany, 2016; pp. 469–477.
160. Zhang, D.; Kim, J.; Crego, J.; Senellart, J. Boosting neural machine translation. *arXiv* **2016**, arXiv:1612.06138.
161. Lyu, Y.; Tsang, I.W. Curriculum loss: Robust learning and generalization against label corruption. *arXiv* **2019**, arXiv:1905.10045.
162. Wu, L.; Tian, F.; Xia, Y.; Fan, Y.; Qin, T.; Lai, J.; Liu, T.-Y. Learning to teach with dynamic loss functions. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 8–14 December 2018*; Volume 31.
163. Li, X.; Wen, L.; Deng, Y.; Feng, F.; Hu, X.; Wang, L.; Fan, Z. Graph neural network with curriculum learning for imbalanced node classification. *arXiv* **2022**, arXiv:2202.02529.
164. Yao, X.; Feng, X.; Han, J.; Cheng, G.; Li, K. Automatic weakly supervised object detection from high spatial resolution remote sensing images via dynamic curriculum learning. *IEEE Trans. Geosci. Remote Sens.* **2020**, *59*, 675–685. [\[CrossRef\]](#)
165. Saxena, S.; Tuzel, O.; DeCoste, D. Data parameters: A new family of parameters for learning a differentiable curriculum. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems, Montreal, QC, Canada, 3–8 December 2019*; Volume 32.
166. Korbar, B.; Tran, D.; Torresani, L. Cooperative learning of audio and video models from self-supervised synchronization. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 8–14 December 2018*; Volume 31.
167. Wang, X.; Chen, Y.; Zhu, W. A survey on curriculum learning. *IEEE Trans. Pattern Anal. Mach. Intell.* **2021**, *44*, 4555–4576. [\[CrossRef\]](#)
168. Soviany, P.; Ionescu, R.T.; Rota, P.; Sebe, N. Curriculum learning: A survey. *Int. J. Comput. Vis.* **2022**, *130*, 1526–1565. [\[CrossRef\]](#)
169. Chang, H.S.; Learned-Miller, E.; McCallum, A. Active bias: Training more accurate neural networks by emphasizing high variance samples. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017*; Volume 30.
170. Amiri, H. Neural self-training through spaced repetition. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, (Long and Short Papers), Minneapolis, MN, USA, 2–7 June 2019*; Volume 1, pp. 21–31.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.